A Project Report on

# VaultShield: Sandboxed, Monitored Execution and Human Review for AI Coding Agents

Submitted in partial fulfillment of the requirements for the award of the degree of

**Bachelor of Engineering**

in

**Computer Science and Engineering (Data Science)**

by

**Aayush Subhash Balip (24207010)**
**Krish Mahesh Patil (24207006)**
**Muhammad Anis Momin (24207008)**
**Sameer Hasan Saiyed (24207014)**

Under the Guidance of

**Prof. Rajashri Chaudhari**
**Prof. Hardiki Achrekar**

**Computer Science and Engineering (Data Science)**
A.P. Shah Institute of Technology
G.B. Road, Kasarvadavali, Thane (W) – 400615
UNIVERSITY OF MUMBAI

**Academic Year 2026–2027**

# Approval Sheet

This Project Report entitled ***VaultShield: Sandboxed, Monitored Execution and Human Review for AI Coding Agents*** submitted by **Aayush Subhash Balip** (24207010), **Krish Mahesh Patil** (24207006), **Muhammad Anis Momin** (24207008), and **Sameer Hasan Saiyed** (24207014) is approved for the partial fulfillment of the requirement for the award of the degree of **Bachelor of Engineering in Computer Science and Engineering (Data Science)** from **University of Mumbai**.

Prof. Rajashri Chaudhari

Guide

Prof. Hardiki Achrekar

Co-Guide

Dr. Pravin Adivarekar

HOD, Computer Science and Engineering (Data Science)

Place: A.P. Shah Institute of Technology, Thane

Date:

# CERTIFICATE

This is to certify that the project entitled ***VaultShield: Sandboxed, Monitored Execution and Human Review for AI Coding Agents*** submitted by **Aayush Subhash Balip** (24207010), **Krish Mahesh Patil** (24207006), **Muhammad Anis Momin** (24207008), and **Sameer Hasan Saiyed** (24207014) for the partial fulfillment of the requirement for award of a degree of **Bachelor of Engineering in Computer Science and Engineering (Data Science)** to the University of Mumbai, is a bonafide work carried out during academic year 2026–2027.

| | |
|---|---|
| **Prof. Rajashri Chaudhari** | **Prof. Hardiki Achrekar** |
| Guide | Co-Guide |
| **Dr. Pravin Adivarekar** | **Dr. Uttam D. Kolekar** |
| HOD, C.S.E (Data Science) | Principal |

| External Examiner(s) | Internal Examiner(s) |
|---|---|
| 1. | 1. |
| 2. | 2. |

Place: A.P. Shah Institute of Technology, Thane

Date:

# Acknowledgement

We express our sincere gratitude to our project guide, **Prof. Rajashri Chaudhari**, for the constant guidance, encouragement and critical feedback provided throughout this project. The guide's insistence on precision, both in engineering and in stating the limits of what had been built, shaped the security model at the heart of this work.

We are equally grateful to our co-guide, **Prof. Hardiki Achrekar**, for valuable suggestions on system design and evaluation, and for the time spent reviewing our progress at every stage.

We thank **Dr. Pravin Adivarekar**, Head of the Department of Computer Science and Engineering (Data Science), for providing the facilities and the supportive environment that made this project possible, and **Dr. Uttam D. Kolekar**, Principal, A. P. Shah Institute of Technology, for the institutional support extended to us.

We also thank the project coordinators for organising the reviews and keeping us on schedule, and all the teaching and non-teaching staff of the department for their help throughout the academic year.

Finally, we thank our colleagues and friends for their feedback on the dashboard and the documentation, and our families for their patience and encouragement.

**Aayush Subhash Balip**
(24207010)

**Krish Mahesh Patil**
(24207006)

**Muhammad Anis Momin**
(24207008)

**Sameer Hasan Saiyed**
(24207014)

## Declaration

We declare that this written submission represents our ideas in our own words, and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented, fabricated or falsified any idea, data, fact or source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Signature)
Aayush Subhash Balip (24207010)

(Signature)
Krish Mahesh Patil (24207006)

(Signature)
Muhammad Anis Momin (24207008)

(Signature)
Sameer Hasan Saiyed (24207014)

Date:

## Abstract

AI coding agents now read repositories, edit many files and execute shell commands with little human intervention. Run directly on a developer's machine, such an agent inherits the developer's privileges: it can read real secrets, write anywhere, contact any host and leave unreviewed changes in version control. Current tooling conflates running an agent with trusting its output. This project presents VaultShield, a pipeline that separates the two. VaultShield runs the agent in a disposable Docker container holding a fresh clone of the repository and a read-only credential file whose values mirror the shape of the developer's .env but contain no real secret; a repository-host token, when needed, is confined to the single git clone process. Terminal output, in-container file changes and policy violations are merged into a timestamped JSON Lines event log. Writes to off-limits paths fail the run, and an optional egress policy routes traffic through a hardened proxy that blocks and records non-allowlisted hosts. A separate hardened helper container then rebuilds the repository from a pre-agent base and commits the change to a dedicated branch with a structured diff, so the agent cannot falsify the evidence. Artifacts are uploaded to a Django REST Framework backend on MongoDB, scanned asynchronously, and approved or rejected by a reviewer in a React dashboard. Only an approved run can become a pull or merge request. The system comprises about 18,100 logical lines of product code. Of 2,704 automated tests, including Docker-backed isolation, egress, token-hygiene and tamper-resistance tests and concurrency tests against a real MongoDB replica set, 2,702 passed; the two failures are confined to the optional interactive sign-in transport. In a benchmark of 40 VaultShield runs, plus 20 direct-host baseline runs, that used a deterministic mock agent rather than a real model-backed agent, VaultShield added 1.17 seconds of fixed overhead per run by default and 1.98 seconds with egress allowlisting; the host checkout was untouched and the change was captured in every run, with no container or network left behind. These figures measure the pipeline's own overhead and are not a measurement of real agents. The Agentic AI Orchestrator of Objective 2 is designed in this report and scheduled for implementation in Semester VIII, together with stronger sandbox isolation, improved monitoring and role-based approval. The evaluation also exposed limits: the shared-kernel boundary of containers, opt-in egress control, a polling file watcher that missed 20% of very short writes, and an approval gate that is a workflow control rather than a separation-of-duties control in the single-user MVP. VaultShield shows that containment, evidence and human review for coding agents can be delivered together at a small fixed pipeline cost in the mock-agent benchmark, with every residual risk stated explicitly; real-agent performance remains to be measured.



**Keywords:** AI coding agents, sandboxing, Docker, human-in-the-loop review, audit logging, credential isolation, egress control, software supply-chain security.

# Contents


**1 Introduction** **1**
- 1.1 Motivation . . . 1
- 1.2 Problem Statement . . . 2
- 1.3 Objectives . . . 2
- 1.4 Scope . . . 3

**2 Literature Review** **4**
- 2.1 Comparative Analysis of Recent Studies . . . 6
- 2.2 Existing AI Coding-Agent Security Solutions . . . 7
- 2.3 Research Gap . . . 7b

**3 Project Design** **8**
- 3.1 Existing System . . . 8
- 3.2 Proposed System . . . 9
- 3.3 Critical Components of System Architecture . . . 10
  - 3.3.1 Sandbox and Container Manager . . . 10
  - 3.3.2 Run Monitoring and Logging . . . 10
  - 3.3.3 Diff Capture on a Dedicated Branch . . . 10
  - 3.3.4 Review and Human Approval Service . . . 12
  - 3.3.5 Backend API and Dashboard . . . 13
  - 3.3.6 Supporting Module: Protect . . . 13
- 3.4 System Diagrams . . . 15
  - 3.4.1 UML Diagram . . . 15
  - 3.4.2 Activity Diagram . . . 17
  - 3.4.3 Use Case Diagram . . . 18
  - 3.4.4 Sequence Diagram . . . 19
- 3.5 Proposed Agentic AI Orchestrator (Objective 2, Semester VIII) . . . 19a
- 3.6 Threat Model . . . 19c

**4 Project Planning and Cost Estimation** **20**
- 4.1 Project Development Methodology . . . 20
- 4.2 Project Schedule and Gantt Chart . . . 20
- 4.3 Team Roles and Responsibilities . . . 21
- 4.4 Software Size Estimation . . . 22
  - 4.4.1 KLOC-Based Estimation . . . 22
- 4.5 Effort Estimation Using COCOMO . . . 22
  - 4.5.1 Basic COCOMO Model . . . 22
  - 4.5.2 Calculation . . . 23
- 4.6 Human Resource Cost . . . 23
- 4.7 Hardware, Software and Cloud Infrastructure Cost . . . 23
- 4.8 Testing, Deployment and Maintenance Cost . . . 24
- 4.9 Total Estimated Project Cost . . . 24
- 4.10 Assumptions and Cost Justification . . . 24
- 4.11 Risk Assessment and Mitigation . . . 25

**5 Project Implementation** **26**
- 5.1 Code Snippets . . . 26

5.1.1 Launching the Sandbox Container . . . 26
5.1.2 One-Shot Credential for Private Clones . . . 27
5.1.3 Event Logging and Off-Limits Enforcement . . . 28
5.1.4 Egress Allow/Deny Decision . . . 29
5.1.5 Branch Commit and Structured Diff . . . 30
5.1.6 Approval State Machine . . . 31
5.2 Steps to Access the System . . . 32
5.3 System Implementation . . . 33
5.3.1 Command-Line Interface . . . 33
5.3.2 Landing Page and Projects Dashboard . . . 33
5.3.3 Run History . . . 34
5.3.4 Run Detail Views . . . 35
5.3.5 Approval Decision and Submission . . . 37
5.3.6 Implementation Summary . . . 37
5.4 Timeline Sem VII . . . 38
5.5 Semester VII Implementation and Semester VIII Plan . . . 38a

**6 Testing** **40**
6.1 Software Testing . . . 40
6.2 Types of Testing . . . 40
6.2.1 Functional Testing . . . 40
6.2.2 System Testing . . . 41
6.3 Testing Results and Findings . . . 41
6.4 Sample Test Cases . . . 42
6.5 Conclusion on Testing . . . 42
6.6 Test Configuration and Analysis of the Two Failed Tests . . . 43

**7 Result and Discussions** **45**
7.1 Introduction . . . 45
7.2 Experimental Setup . . . 45
7.3 Evaluation Metrics . . . 46
7.4 Performance Analysis . . . 46
7.5 Overhead with and without VaultShield . . . 48
7.6 Safety Events Captured . . . 48
7.7 Secret Exposure With and Without Protection . . . 49
7.8 Execution Evidence from a Live Run . . . 49
7.9 Comparison with Existing Approaches . . . 52
7.10 Case Study: Analysis of Real Runs . . . 53
7.11 Overall Discussion and Limitations . . . 54
7.12 Chapter Summary . . . 54

**8 Conclusion** **55**

**9 Future Scope** **56**
9.1 Semester VIII Enhancements . . . 56
9.2 Further Future Work . . . 56

**Bibliography** **57**

# List of Figures


3.1 Existing workflow – direct agent execution on the developer's host . . . . 8
3.2 Proposed VaultShield pipeline . . . . . 11
3.3 Per-run egress network topology . . . . . 11
3.4 Run status state machine . . . . . 12
3.5 Overall system architecture . . . . . 14
3.6 UML class diagram of the core backend entities and CLI classes . . . . . 16
3.7 Activity diagram of a VaultShield run, from agent kickoff to PR creation 17
3.8 Use case diagram . . . . . 18
3.9 Sequence diagram of one complete approved run . . . . . 19
3.10 Proposed workflow of the Agentic AI Orchestrator (Semester VIII) . . . . 19a
3.11 Threat model: trust zones and attack paths . . . . . 19d

5.1 Sandbox container creation and teardown . . . . . 27
5.2 Authenticated clone with a one-shot credential helper . . . . . 28
5.3 Event records, off-limits matching and the JSON Lines writer . . . . . 29
5.4 Egress decision for one request target . . . . . 30
5.5 Committing the run branch and computing the diff . . . . . 31
5.6 Transactional approve/reject decision . . . . . 32
5.7 Terminal output of a successful vaultshield run with the mock agent . . . 33
5.8 Landing page . . . . . 34
5.9 Projects dashboard . . . . . 34
5.10 Run history with status filters . . . . . 35
5.11 Run detail – Diff tab . . . . . 35
5.12 Run detail – Timeline tab . . . . . 36
5.13 Run detail – Screenshots tab . . . . . 36
5.14 Run detail – Security tab . . . . . 37
5.15 Run detail after approval, showing the submit command . . . . . 37
5.16 Gantt chart of the VaultShield major project . . . . . 39

6.1 Re-run of the three automated suites on 1 November 2026 . . . . . 43
6.2 The two failing tests re-run individually, with their durations . . . . . 44

7.1 Mean stage latency in the default configuration . . . . . 47
7.2 Mean end-to-end time by configuration . . . . . 48
7.3 A real vaultshield run with the egress allowlist and the mock agent . . . 50
7.4 Run artifacts: dummy credential file, event log and structured diff . . . . 50
7.5 Live dashboard: Timeline tab of the real run . . . . . 51
7.6 Live dashboard: Diff tab of the real run, awaiting review . . . . . 51
7.7 Approval gate refusing an unapproved run, and no leftover containers or host changes . . . . . 51
7.8 Isolation probe run inside a real sandbox . . . . . 52
7.9 Docker-backed egress, token-hygiene and tamper-resistance tests . . . . . 52

# List of Tables


2.1 Comparative analysis of the reviewed literature . . . . . 6
2.2 Comparison with existing AI coding-agent security solutions . . . . . 7a

3.1 Comparison of the existing system and the proposed system . . . . . 9
3.2 Planned components of the Agentic AI Orchestrator . . . . . 19b
3.3 Threat model: attacks, mitigations and residual risk . . . . . 19e

4.1 Planned project schedule for VaultShield . . . . . 21
4.2 Team roles and responsibilities . . . . . 22
4.3 Measured software size by component . . . . . 22
4.4 Basic COCOMO estimate . . . . . 23
4.5 Actual human-resource cash expenditure . . . . . 23
4.6 Actual hardware, software and infrastructure cash expenditure . . . . . 24
4.7 Actual testing, deployment and maintenance cash expenditure . . . . . 24
4.8 Total actual project cash expenditure . . . . . 24
4.9 Risk assessment and mitigation . . . . . 25

5.1 Features implemented in Semester VII . . . . . 38a
5.2 Enhancements planned for Semester VIII . . . . . 38a

6.1 Types of testing performed . . . . . 40
6.2 Automated test suite results . . . . . 41
6.3 Issues found and their resolution . . . . . 41
6.4 Sample test cases . . . . . 42
6.5 Test environment and configuration . . . . . 43

7.1 Benchmark configurations (all with the mock agent) . . . . . 45
7.2 Evaluation metrics . . . . . 46
7.3 Stage latency in the default configuration . . . . . 47
7.4 End-to-end latency by configuration . . . . . 47
7.5 Safety events and evidence captured . . . . . 48
7.6 Secret exposure under four protection configurations . . . . . 49
7.7 Outcome of the same threats under three approaches . . . . . 53
7.8 Residual risks and limitations . . . . . 54

8.1 Achievement of objectives . . . . . 55

# List of Abbreviations

| | | |
|---|---|---|
| AA | : | Level AA conformance of the Web Content Accessibility Guidelines |
| ACI | : | Agent-Computer Interface |
| API | : | Application Programming Interface |
| ASGI | : | Asynchronous Server Gateway Interface |
| CI/CD | : | Continuous Integration / Continuous Delivery |
| CLI | : | Command-Line Interface |
| COCOMO | : | Constructive Cost Model |
| CORS | : | Cross-Origin Resource Sharing |
| CPU | : | Central Processing Unit |
| CWE | : | Common Weakness Enumeration |
| DNS | : | Domain Name System |
| DRF | : | Django REST Framework |
| HTTP | : | Hypertext Transfer Protocol |
| HTTPS | : | Hypertext Transfer Protocol Secure |
| IDE | : | Integrated Development Environment |
| IP | : | Internet Protocol |
| JSON | : | JavaScript Object Notation |
| JSONL | : | JSON Lines |
| KLOC | : | Thousand Lines of Code |
| LLM | : | Large Language Model |
| LM | : | Language Model |
| MR | : | Merge Request |
| MVP | : | Minimum Viable Product |
| OAuth | : | Open Authorization |
| OCR | : | Optical Character Recognition |
| PM | : | Person-Month |
| PR | : | Pull Request |
| PTY | : | Pseudo-Terminal |
| RBAC | : | Role-Based Access Control |
| REST | : | Representational State Transfer |

| | | |
|---|---|---|
| SHA | : | Secure Hash Algorithm |
| SLOC | : | Source Lines of Code |
| SNI | : | Server Name Indication |
| SPA | : | Single-Page Application |
| SQL | : | Structured Query Language |
| STRIDE | : | Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege |
| TLS | : | Transport Layer Security |
| TTL | : | Time To Live |
| UI | : | User Interface |
| UML | : | Unified Modeling Language |
| URL | : | Uniform Resource Locator |
| UTC | : | Coordinated Universal Time |
| VCS | : | Version Control System |
| VM | : | Virtual Machine |
| WCAG | : | Web Content Accessibility Guidelines |
| WSGI | : | Web Server Gateway Interface |

# Chapter 1

# Introduction

Large Language Models trained on source code have moved from completing a single line to operating as autonomous coding agents that read a repository, plan a change, edit many files, run shell commands and iterate on test results with little human intervention. Tools such as OpenAI Codex CLI, Google Antigravity, Amazon's Kiro and Anthropic's Claude Code expose this capability directly in the developer's terminal or editor, and research systems such as SWE-agent [3] and OpenHands [4] resolve a meaningful fraction of real GitHub issues from benchmarks like SWE-bench [2].

The same capability makes coding agents hazardous. An agent is an ordinary process that runs with the privileges of the user who launched it. On a developer's workstation it can read every secret in the shell environment and in nearby .env files, write anywhere the user can write, run commands the prompt never asked for, contact arbitrary hosts and leave changes in version control before anyone has examined them. Prior work shows that LLM-generated code is often insecure [8], that LLM-integrated applications can be hijacked through indirect prompt injection [7], and that even carefully aligned agents take risky actions in a non-trivial fraction of high-stakes scenarios [5]. Governance guidance recommends constraining an agent's action space, requiring human approval for consequential actions and making agent activity legible [9], [10], but these recommendations are rarely packaged as a pipeline a developer can run against an existing repository.

VaultShield addresses that gap. It runs an AI coding agent inside a disposable Docker container, records what the agent does, captures the resulting code changes as a reviewable Git diff on a dedicated branch, and holds those changes behind an explicit human approval step before they can become a pull request or merge request. The developer's working tree, real credentials and repository write access are never handed to the agent. The system consists of a Python command-line interface that orchestrates the sandboxed run, a per-run Docker execution plane, and a Django REST Framework backend with a React dashboard where a reviewer approves or rejects the run. Producing evidence therefore never implies accepting it.

## 1.1 Motivation

The first motivation is the asymmetry between the cost of an agent mistake and the cost of preventing it. An agent that deletes the wrong directory or commits a hard-coded key can cost hours of recovery, or far more if credentials leak, while VaultShield's throwaway-container pipeline added about 1.2 seconds of fixed overhead per run in a mock-agent benchmark.

The second motivation is credential exposure. Applications keep database strings, API keys and session secrets in a local .env file inside the very repository the agent is asked to modify. A preliminary spike in this project confirmed the risk: with no protection, a production coding agent asked to reveal a value from .env read the file with its built-in tool and printed both test secrets. Agent-specific deny rules reduced but did not remove the exposure, and only keeping the real value off disk proved reliable.

The third motivation is reviewability. Code review is the primary quality-control mechanism of modern software engineering [14], yet agent output often bypasses the pull-request workflow because it lands directly in the working tree. Capturing agent output on a dedicated branch, together with a timestamped event log and screenshots, lets a reviewer apply the same review process to agent changes and leaves a durable audit trail.

The fourth motivation is honest security engineering. Container isolation shares the host kernel and is not equivalent to a virtual machine boundary [11], [12], [13]. VaultShield therefore states which controls it implements and which risks remain, so that adopters can judge where it fits in their own threat model.

## 1.2 Problem Statement

Given a Git repository, a natural-language prompt and an AI coding agent, the problem is to let the agent attempt the requested change while guaranteeing that it never executes against the developer's real working tree, real application secrets or repository-host credentials; that everything it does is recorded as reviewable evidence; that its change is captured in a form it could not have tampered with; and that no change reaches a shared branch or pull request without an explicit human approval decision.

The agent must still be able to work, so it needs a real repository, a functioning toolchain and sometimes its own model API credential. The application must be able to start inside the sandbox for screenshots, so configuration values must keep their structural shape even though their content is fake. Every container, network and temporary branch must be removed on every exit path, and the pipeline overhead must stay small compared with typical agent run times.

## 1.3 Objectives

The project set out to achieve the following objectives.

1. Develop a universal AI agent execution platform compatible with multiple AI coding agents and development environments through a common CLI interface.
2. Implement an intelligent Agentic AI Orchestrator that understands project context, asks clarification questions before execution, and provides enriched context to AI agents to improve accuracy and reduce hallucinations.
3. Ensure secure AI-assisted software development by executing AI agents inside isolated Docker-based sandbox environments while protecting production credentials and resources.
4. Provide comprehensive monitoring and automated security analysis by capturing AI activities, execution logs, code changes, and identifying security vulnerabilities or unsafe operations.
5. Enable transparent governance and human approval through a web-based dashboard that allows developers to review, verify, approve, or reject AI-generated changes before deployment.

## 1.4 Scope

The scope of the VaultShield project covers the sandboxed execution, monitoring and human review of AI coding agents working on a Git repository.

- **Command-Line Interface:** the vaultshield CLI with the init, login, run, submit and teardown commands and the supporting protect command group.
- **Sandboxed Execution:** a per-run Docker sandbox with a fresh clone, a read-only dummy credential file and guaranteed teardown.
- **Agent Adapters:** the mock, codex, kiro and antigravity agents behind a common adapter interface.
- **Monitoring and Egress Control:** terminal capture, in-container file watching, off-limits paths, a JSON Lines event log and an optional egress policy enforced by a hardened proxy.
- **Trusted Change Capture:** the run branch and structured diff built in a separate hardened helper container, with run outputs scanned for leaked credentials.
- **Backend, Dashboard and Submission:** a Django REST Framework backend on MongoDB with the run state machine and Celery-based scanning, a React review dashboard, and pull or merge request submission to GitHub or GitLab for approved runs.

Objective 2, the Agentic AI Orchestrator, is not implemented in Semester VII; its proposed design is given in Section 3.5. The following are outside the Semester VII scope; the Orchestrator, stronger isolation, improved monitoring and role-based approval are planned for Semester VIII (Section 9.1):

- **Stronger Isolation and Hosted Execution:** virtual-machine-grade isolation and running the CLI as a hosted service; container escape is a documented residual risk.
- **Behavioural Judgement:** automatic analysis of agent intent, content-level network filtering and detection of deliberately encoded exfiltration.
- **Multi-User Operation:** multi-tenant operation, role-based access control and separate runner and reviewer identities.
- **Other Integrations:** live output streaming, other repository hosts, automatic merging and CI/CD pipeline integration.

The minimum viable product is scoped to a single developer's machine, so the approval gate is a workflow control within one trust domain rather than a security boundary between mutually distrustful parties.

# Chapter 2

# Literature Review

The literature relevant to VaultShield falls into five overlapping strands: the capabilities and evaluation of autonomous coding agents, the security risks of LLM agents and of the code they generate, the governance and visibility of agentic systems, isolation technologies for running untrusted code, and human code review. Each strand is reviewed below, followed by a comparative analysis and a statement of the research gap that the project addresses.

Code-generating models and the need for sandboxes. Chen et al. [1] introduced Codex, a GPT model fine-tuned on public GitHub code, together with the HumanEval benchmark of hand-written programming problems scored by functional correctness. Because scoring functional correctness requires executing model-generated programs, the authors ran those programs inside a gVisor-based sandbox and discussed misuse, over-reliance and insecure code as broader risks. The paper established, at the start of the code-LLM era, that model output must be treated as untrusted code and executed only behind an isolation boundary. Its sandbox, however, was an internal evaluation harness for short, self-contained functions rather than a developer-facing tool for repository-scale changes with human review.

Repository-scale agents. Jimenez et al. [2] proposed SWE-bench, 2,294 task instances drawn from real GitHub issues and pull requests across twelve popular Python repositories, in which a model must produce a patch that makes the repository's failing tests pass. SWE-bench moved evaluation from isolated functions to whole repositories, and the best model of the time resolved fewer than 2% of the issues. Yang et al. [3] then showed with SWE-agent that a purpose-built agent-computer interface, with dedicated commands for viewing, searching and editing files, raises this to 12.5% with GPT-4 Turbo, executing inside a Docker container. Wang et al. [4] presented OpenHands, an open platform in which agents act through a sandboxed Docker runtime and every action and observation is recorded in an event stream. These systems validate two of VaultShield's design choices: a container is the practical execution environment for repository-scale agents, and a chronological record of agent actions is the natural unit of observability. Their sandboxes, however, exist to give the agent a reproducible workspace. None of them holds the agent's patch for human approval, and none keeps the developer's real secrets out of the workspace.

Risks of LLM agents. Ruan et al. [5] introduced ToolEmu, which uses a language model to emulate tool execution so that agents can be red-teamed against high-stakes scenarios without real side effects, together with an automatic safety evaluator. Across 36 toolkits and 144 test cases, even the safest agent they evaluated showed risky failures in 23.9% of cases. Debenedetti et al. [6] built AgentDojo, a dynamic environment with 97 realistic tasks and 629 security test cases for measuring prompt-injection attacks against tool-using agents and the defences against them, and found that both remain far from solved. Greshake et al. [7] demonstrated indirect prompt injection, in which instructions planted in data that an LLM-integrated application retrieves take control of the application and can cause data exfiltration. For coding agents the implication is direct, because a repository's own files, issues and dependencies are untrusted input.

These works characterise the threat convincingly, but they are evaluation frameworks and do not provide an operational containment and review pipeline for developers.

Security of AI-generated code. Pearce et al. [8] prompted GitHub Copilot with 89 scenarios based on MITRE's top Common Weakness Enumeration categories and found that about 40% of the 1,689 generated programs contained vulnerabilities. This result motivates VaultShield's static security scan over every ingested diff and, more fundamentally, its requirement that a human reviews agent code before it is merged. The study evaluated completion suggestions rather than autonomous agents, so it does not cover agents that also execute commands; VaultShield extends the concern from insecure code to insecure behaviour.

Governance and visibility of agentic systems. Shavit et al. [9] proposed practices for keeping agentic AI systems safe and accountable, including evaluating suitability for a task, constraining the action space, requiring human approval for high-impact actions, setting safe defaults, making agent activity legible, automatic monitoring, attributability and interruptibility. Chan et al. [10] argued that oversight of AI agents depends on visibility and identified three categories of measures: agent identifiers, real-time monitoring and activity logging. VaultShield can be read as a concrete implementation of several of these practices for coding agents. The run identifier names the sandbox, the branch, the run directory and the backend record; the event log, egress log and screenshots form the activity log; and the approval state machine is the human-approval control. Both papers are deliberately technology-neutral, so they leave the engineering of such controls as an open problem.

Container isolation and its limits. Combe et al. [11] analysed Docker from a security perspective and identified the main areas of concern: isolation through Linux namespaces and control groups, the shared host kernel, network configuration and the trustworthiness of the image distribution chain. They concluded that Docker's default configuration offers reasonable isolation but that its security depends heavily on configuration. Agache et al. [12] described Firecracker, the virtual machine monitor behind AWS Lambda, whose microVMs boot in as little as 125 ms with under 5 MiB of memory overhead, showing that VM-grade isolation can approach container-like efficiency. Young et al. [13] measured gVisor, a user-space kernel that intercepts system calls to shrink the host kernel's attack surface, and showed that this stronger isolation carries substantial costs for system-call-heavy and file-system-heavy workloads. Together these papers frame VaultShield's trade-off: Docker containers are cheap and widely available but share the kernel, while gVisor and microVMs offer stronger boundaries at a cost in performance or operational complexity. VaultShield uses Docker for its MVP, documents the escape risk and lists gVisor and microVMs as future work.

Human code review. Bacchelli and Bird [14] studied modern code review at Microsoft through observation, interviews and surveys. Although finding defects is the main stated motivation for review, much of its actual value lies in knowledge transfer, team awareness and alternative solutions, and the main challenge for reviewers is understanding the change and its context. This finding shapes VaultShield's dashboard: the reviewer receives not only the diff but also the timeline of what the agent did, screenshots of the running application and automated findings, so that understanding an agent's change does not require reconstructing its reasoning from the patch alone.

## 2.1 Comparative Analysis of Recent Studies

Table 2.1 compares the reviewed works and records the limitation of each that VaultShield addresses.

**Table 2.1:** Comparative analysis of the reviewed literature

| Ref. | Approach | Limitation relative to this project |
|---|---|---|
| [1] Chen et al., 2021 | Code LLM evaluated by running generated programs in a sandbox | Internal harness for short functions; no repository workflow, credential model or review |
| [2] Jimenez et al., 2024 | Repository-level benchmark from real issues | Measures capability; patches are applied for scoring, not held for approval |
| [3] Yang et al., 2024 | Agent with a purpose-built agent-computer interface in Docker | Container is a workspace, not a trust boundary; no secret substitution, egress control or review gate |
| [4] Wang et al., 2025 | Open agent platform with an event stream and sandboxed runtime | Changes are not captured on a reviewable branch behind an approval state machine |
| [5] Ruan et al., 2024 | LM-emulated sandbox for red-teaming agents | Emulation cannot contain a real agent acting on a real repository |
| [6] Debenedetti et al., 2024 | Benchmark for prompt-injection attacks and defences | Evaluates attacks; offers no containment or audit pipeline |
| [7] Greshake et al., 2023 | Indirect prompt injection via retrieved data | Characterises the threat; mitigation is left open |
| [8] Pearce et al., 2022 | Security study of AI code suggestions | Studies suggestions, not agents that also execute commands |
| [9] Shavit et al., 2023 | Governance practices for agentic AI | Technology-neutral; no implementation for coding agents |
| [10] Chan et al., 2024 | Visibility measures for AI agents | Defines what to log, not how to capture tamper-resistant evidence |
| [11] Combe et al., 2016 | Security analysis of Docker | Does not address agent credentials or review |
| [12] Agache et al., 2020 | Firecracker microVM isolation | Requires KVM; not integrated with a review workflow |
| [13] Young et al., 2019 | Performance study of gVisor | Quantifies isolation cost; is not an agent pipeline |
| [14] Bacchelli & Bird, 2013 | Empirical study of modern code review | Concerns human changes; agent changes need behavioural context |

## 2.2 Existing AI Coding-Agent Security Solutions

The research systems above are complemented by security features that agent vendors and platform providers have shipped. These products are the practical alternatives a developer would compare VaultShield against, so they are reviewed here from their official documentation, accessed on 1 November 2026.

Agent-native OS sandboxes. Claude Code runs the shell commands its agent issues inside an operating-system sandbox (Seatbelt on macOS, bubblewrap on Linux) that limits the files and network domains those commands can reach [16], and Anthropic notes that effective sandboxing needs both filesystem and network isolation [17]. The OpenAI Codex CLI similarly enforces an OS-level sandbox whose default limits writes to the active workspace and disables network access, combined with an approval policy that decides when the agent must ask before acting [18]. Both are strong per-command controls, but they are built into one vendor's agent, they protect the developer's real checkout in place rather than a copy, the real .env inside the workspace remains readable by the agent's file tools unless separately denied, and the resulting edits land directly in the working tree with no held review step. Our feasibility spike (Section 7.7) found that agent-level deny rules alone did not stop a production agent from reading a secret file.

Hosted agent environments. Codex cloud runs tasks in OpenAI-managed containers with a setup phase that may use the network and an agent phase that is offline by default; configured secrets are available only during setup [18]. The GitHub Copilot cloud agent works in an ephemeral, firewalled GitHub Actions environment, receives only secrets placed in a dedicated copilot environment, can push only to its own branch, and its pull requests run CodeQL and secret scanning; workflows triggered by its pull requests need approval from a user with write access [19]. These are the closest existing approaches to VaultShield. They are, however, hosted services tied to one vendor's agent and one repository host, the code leaves the developer's machine, and review happens on a pull request that already exists on the shared host.

Local microVM sandboxes and open agent platforms. Docker Sandboxes run a coding agent in a local microVM with its own kernel, Docker daemon, filesystem and network, so they offer a stronger isolation boundary than an ordinary container; the project directory is shared with the host [20]. OpenHands runs its agent server in a Docker sandbox, records an event stream and offers an optional confirmation mode and security analyser [4], [21]. Neither substitutes dummy credentials for a shared project's .env, builds a change record outside the agent's control or holds the change behind an approval state before it reaches version control.

Table 2.2 compares these solutions with VaultShield on the controls this project targets. A tick records a documented default or built-in feature, "opt." an opt-in or configuration-dependent feature and "–" that the cited documentation does not describe it, which does not prove the feature is absent; the VaultShield column reports the Semester VII implementation, including its weaknesses.

**Table 2.2:** Comparison with existing AI coding-agent security solutions

| Control | Claude Code sandbox [16] | Codex CLI [18] | Copilot cloud agent [19] | Docker Sandboxes [20] | OpenHands [21] | VaultShield |
|---|---|---|---|---|---|---|
| Runs on the developer's machine | ✓ | ✓ | – | ✓ | ✓ | ✓ |
| Works with several vendors' agents | – | – | – | ✓ | ✓ | ✓ |
| Isolation boundary | OS sandbox | OS sandbox | Hosted VM | MicroVM | Container | Container (shared kernel) |
| Host checkout never modified by the agent | – | – | ✓ | – | opt. | ✓ |
| Real .env replaced by dummy values | – | – | Separate secrets | – | – | ✓ |
| Repository token hidden from the agent | – | – | – | – | – | ✓ |
| Outbound network restricted by default | opt. | ✓ | ✓ | opt. | – | opt. (allowlist) |
| Timestamped activity log | – | – | Session log | – | ✓ | ✓ |
| Change record built outside the agent's control | – | – | – | – | – | ✓ |
| Change held for approval before any PR exists | – | – | – | – | – | ✓ |
| Automated scan of the change | – | – | ✓ | – | opt. | ✓ |
| Separate reviewer roles | – | – | ✓ | – | – | Semester VIII |

The table shows that VaultShield is not the strongest solution on every row. Docker Sandboxes and hosted environments provide a stronger isolation boundary than a shared-kernel container, Codex and Copilot restrict the network by default where VaultShield requires opting in, and Copilot already separates the agent from reviewers with repository roles. VaultShield's contribution lies in the rows that no other solution covers together: it keeps the developer's real secrets and checkout away from any vendor's agent, builds the change record outside the agent's reach, and holds the change behind an approval state machine before anything reaches the repository host, while running locally.

## 2.3 Research Gap

The literature shows that coding agents can now make repository-scale changes [2], [3], [4], that their code and actions are risky in measurable ways [5], [6], [7], [8], that container security depends on configuration [11], [12], [13], and that governance research recommends human approval, legible logs and monitoring [9], [10]. The product survey in Section 2.2 shows that industry has adopted parts of these recommendations, but each solution covers a different subset. The specific gaps are the following.

G1. **Containment is tied to one vendor.** OS sandboxes and approval policies are built into individual agents [16], [18], and the hosted environments serve only their own agent [18], [19]. A team that uses several agents has no single control point with one policy and one evidence format.

G2. **Secrets inside the repository are still exposed.** Local sandboxes restrict paths outside the workspace, but the project's own .env sits inside it. Deny rules are agent-specific and, in our spike, brittle; no surveyed local solution replaces the values with structurally valid dummies so the application still starts.

G3. **Evidence is produced by the component being judged.** Event streams and session logs [4], [10] are recorded by, or alongside, the agent's own environment. None of the surveyed solutions rebuilds the change from a trusted base in a separate component, so a hostile agent that tampers with Git state could misrepresent its change.

G4. **Review comes after the change is already in version control.** Governance guidance asks for human approval of consequential actions [9], yet local agents write straight into the working tree and hosted agents push a branch before a human looks at it. No surveyed solution holds the change behind an explicit, race-free approval state before a branch is pushed or a pull request is opened.

G5. **Limits are rarely measured or stated.** Research frameworks evaluate attacks [5], [6] and products document features, but there is little published measurement of what a containment pipeline costs per run and which attacks it does not stop.

VaultShield addresses G1 to G4 with an agent-agnostic adapter interface, shape-preserving dummy credentials and a one-shot clone token, a separate trusted capture helper, and an approval state machine enforced before submission. It addresses G5 by measuring its own overhead and by listing its residual risks, including the areas, such as isolation strength and default-deny networking, where existing products are ahead of it.

# Chapter 3

# Project Design

## 3.1 Existing System

Developers currently run AI coding agents in one of three ways. The most common is raw local execution, where the agent's CLI or editor extension runs in the developer's own checkout with the developer's environment, permissions and network access, and its changes appear as uncommitted edits. Some products add permission prompts or per-tool deny rules, but these are product-specific and often relaxed by users who find repeated prompts tedious. The second practice is ad-hoc containerisation, which usually bind-mounts the real working tree, so every write still lands on the host and the real .env file travels into the container. The third, development containers and cloud workspaces, is designed for convenience rather than for constraining an untrusted actor and holds nothing for review. Figure 3.1 summarises this workflow.

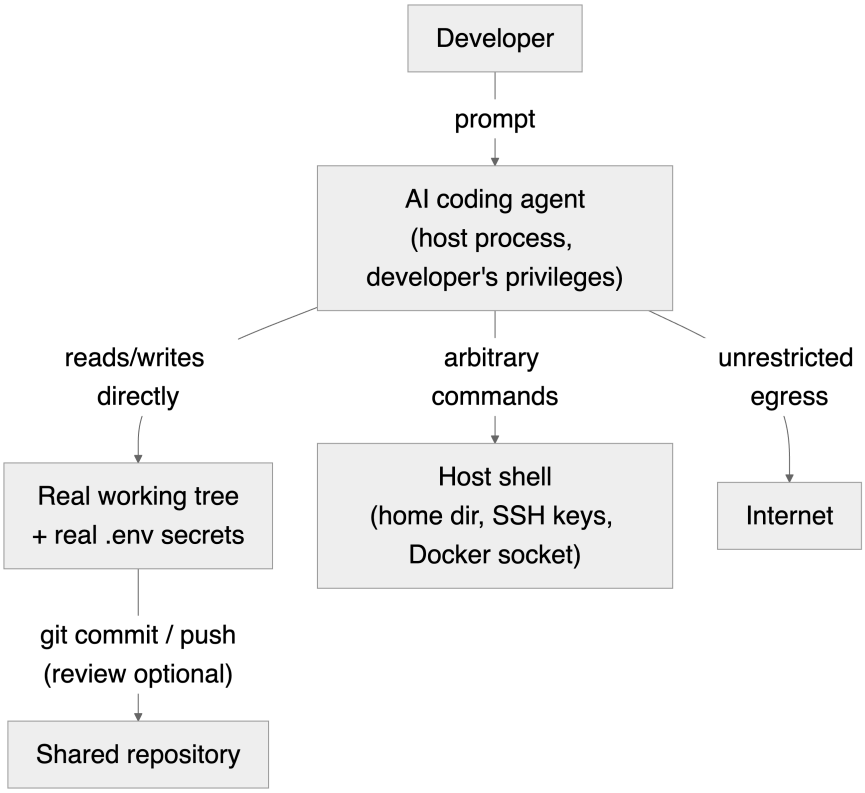



**Figure 3.1:** Existing workflow – direct agent execution on the developer's host

As Figure 3.1 shows, execution and acceptance are a single step and every arrow out of the agent touches a real asset. There is no isolation of side effects, no separation between the agent and the developer's secrets, no structured record of what happened, no enforced review before changes are committed, no automatic cleanup of processes the agent started,

and no record of which hosts the agent contacted.

## 3.2 Proposed System

VaultShield replaces this single step with a pipeline in which execution, evidence collection, storage and review are separate stages. The developer runs vaultshield run with an agent, a prompt and a repository URL. The CLI loads the project configuration, generates a run identifier, creates a per-run directory and registers the run with the backend before any container exists, so that later failures can be recorded. It then writes a dummy credential file that mirrors the shape of the developer's .env and resolves the agent adapter.

The CLI then creates a Docker container, mounts only the dummy credential file, clones the repository inside the container and installs dependencies. If egress control is enabled, the sandbox is moved onto an internal network whose only peer is a filtering proxy. Before any agent code runs, the CLI records the pre-agent commit through the Docker API as a trusted base. The agent runs under a pseudo-terminal while an in-container watcher records file changes, and both streams are merged into a timestamped event log; configured routes are screenshotted with a headless browser. The sandbox is then killed, and a separate hardened capture helper rebuilds the repository from the trusted base and a filtered copy of the work tree, commits the agent's changes to a dedicated run branch and computes the structured diff. Outputs are scanned for the agent's own credentials, the sandbox is torn down and the artifacts are uploaded, moving the run to awaiting_review. A reviewer inspects the evidence in the dashboard and approves or rejects the run, and only an approved run can be submitted as a pull or merge request. Figure 3.2 presents this pipeline.

Table 3.1 contrasts the proposed system with the existing practices.

**Table 3.1:** Comparison of the existing system and the proposed system

| Concern | Existing practice | Proposed system |
|---|---|---|
| Working tree | Real checkout, modified in place | Fresh clone inside the container; host tree never mounted |
| Secrets | Real .env readable by the agent | Shape-preserving dummy values in a read-only .env.sandbox |
| Repository token | Often visible to the agent | Given only to the single git clone process |
| Network | Unrestricted, unlogged | Optional allowlist through a hardened proxy, every decision logged |
| Evidence | Terminal scroll-back | Event log, egress log, screenshots and structured diff |
| Change capture | Uncommitted edits | One commit on a run branch built by a separate helper |
| Review | Optional and informal | Enforced state machine; PR only from approved |

| Concern | Existing practice | Proposed system |
|---|---|---|
| Cleanup | Manual | Guaranteed teardown and a scheduled reaper |

## 3.3 Critical Components of System Architecture

The architecture has three planes: a host-side CLI pipeline, a per-run Docker execution plane, and a backend and dashboard review plane. The critical components are described below.

### 3.3.1 Sandbox and Container Manager

The sandbox manager owns exactly one Docker container per run. It verifies that the credential file exists, creates a container named after the run, mounts the dummy credential file read-only, clones the repository inside the container and installs dependencies. The agent is executed through exec calls rather than as the container's main process, so the container's lifetime bounds every process the agent starts. For private repositories, the repository-host token is passed only as the environment of the single git clone process, so it never appears in the container's environment, the clone's Git configuration or any later process, and it is sent only to hosts on a strict per-provider allowlist. The Docker socket is never mounted.

When egress control is enabled, the EgressGuard builds the per-run topology shown in Figure 3.3: an internal network with no gateway, an ordinary bridge, and a hardened proxy attached to both. The sandbox is moved onto the internal network before the agent runs, so its only reachable peer is the proxy, which checks every destination host against the allowlist and logs each decision.

Any error during creation removes the partial container and raises a single SandboxError whose message never contains a token. Teardown force-removes the container and is called from a finally block, so it runs on success, failure, timeout and interruption alike.

### 3.3.2 Run Monitoring and Logging

The monitor produces the run's event log, events.jsonl, in which each line records an event type, a timestamp and a payload. The three event types are terminal output from the agent's pseudo-terminal, file changes observed by an in-container watcher, and violations such as off-limits writes, egress denials and leaked credentials. Each changed path is checked against the project's off-limits paths, and a match fails the run only after the log has been written, so evidence always outlives the failure. Browser capture starts the application inside the sandbox and takes one screenshot per configured route. Token hygiene scans the diff, the event log and changed files for the agent's own credential bytes; a hit fails the run, quarantines the diff and scrubs uploaded payloads.

### 3.3.3 Diff Capture on a Dedicated Branch

If the branch and diff were built inside the agent's own sandbox, a hostile agent could tamper with Git state to misrepresent its changes. VaultShield therefore reads the pre-agent commit through the Docker API before the agent runs, and after the agent finishes

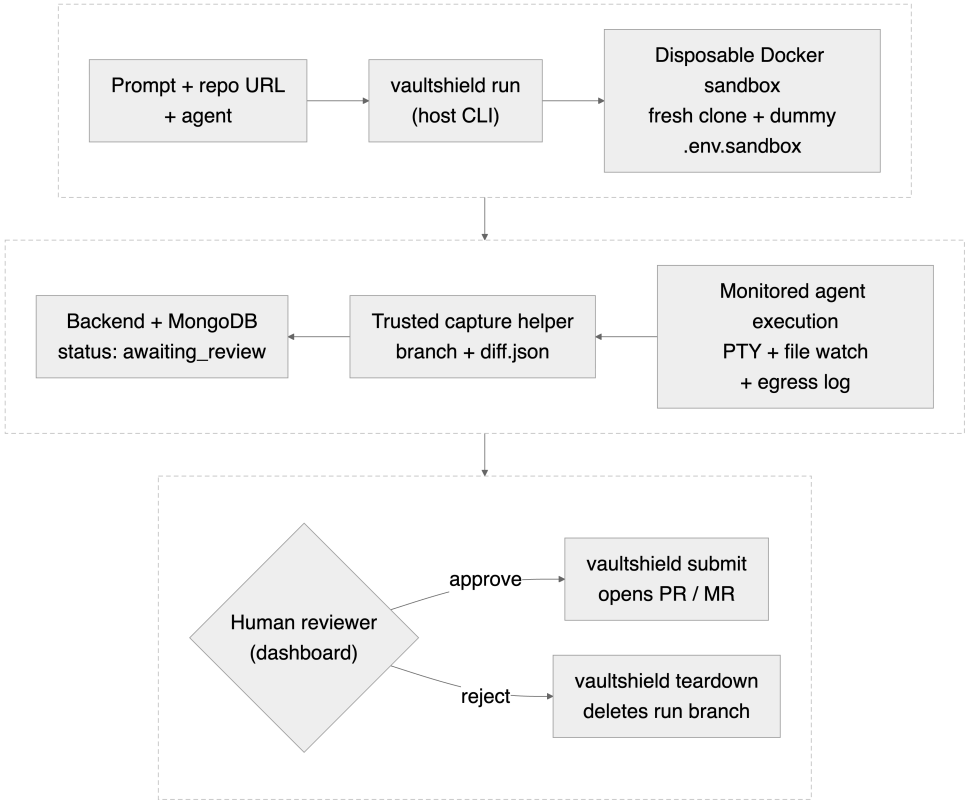



**Figure 3.2:** Proposed VaultShield pipeline

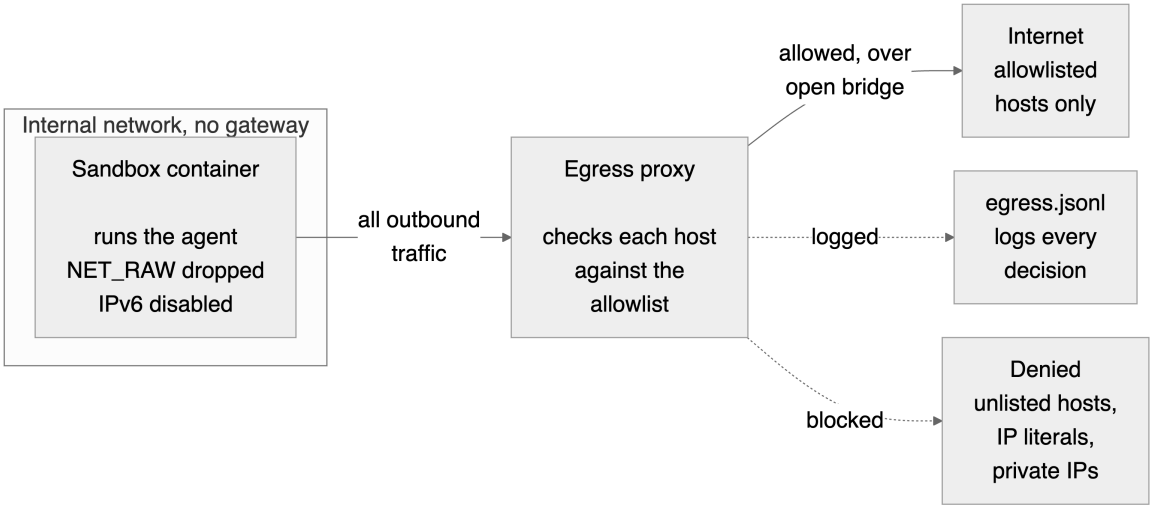



**Figure 3.3:** Per-run egress network topology

it kills the sandbox and extracts a filtered archive of the work tree. A fresh capture helper container with no network, no capabilities and no environment rebuilds the repository from the trusted base, refuses unsafe Git configuration, commits every change to the run branch as exactly one commit, and writes diff.json with per-file additions, deletions and size-capped patches. The branch is exported as a Git bundle because the helper is destroyed at the end of the run.

### 3.3.4 Review and Human Approval Service

The Run.status field is a state machine, shown in Figure 3.4. The CLI may move a run from running to awaiting_review or failed, and the approve and reject endpoints change the status only if it is still awaiting_review, inside a MongoDB transaction that also creates the Approval record. Of two concurrent decisions exactly one succeeds, and the other receives HTTP 409 with the run's actual status.

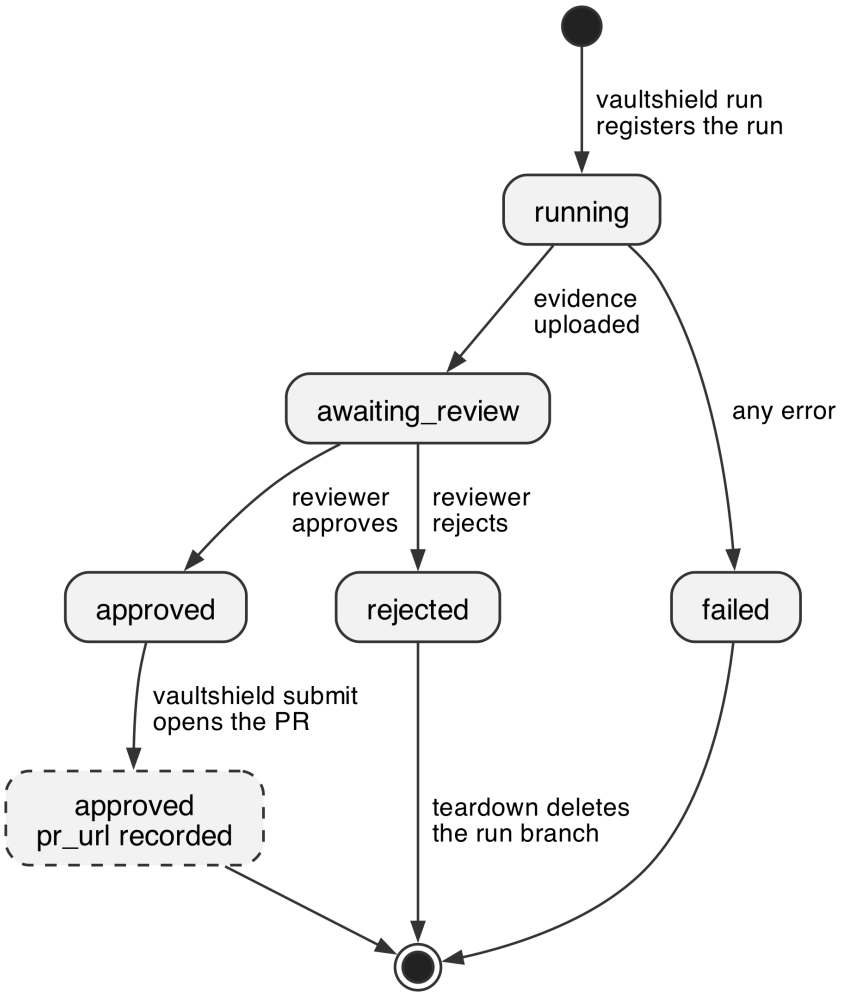



**Figure 3.4:** Run status state machine

vaultshield submit re-reads the run and refuses before any side effect unless its status is approved. For an approved run it checks that the current checkout matches the original repository, imports the branch from the verified bundle, pushes it without force, opens the pull or merge request and records its URL on the run. vaultshield teardown deletes the branch of a rejected run.

### 3.3.5 Backend API and Dashboard

The backend is a Django 5.2 project with Django REST Framework and MongoDB as its system of record. Every API path except login requires token authentication. The CLI registers runs, uploads events, diffs and screenshots, and patches the status; the dashboard reads run details and records decisions. Celery workers with Redis run a static security scan over the added lines of each diff, reap orphaned sandboxes every 15 minutes and flag approved but unsubmitted runs. The dashboard is a React 19 application built with Vite 7 that shows a projects overview, a filterable run history and a run detail page with Diff, Timeline, Screenshots and Security tabs and the decision controls.

### 3.3.6 Supporting Module: Protect

Protect covers agents that developers still run outside the sandbox: it moves every .env value into the macOS Keychain, leaves a placeholder in the file and injects the real values only into the application started through vaultshield exec.

Figure 3.5 shows all components and the data flows between them. The CLI is the only component that talks to the Docker daemon, the dummy credential file is the only host path mounted into the sandbox, artifacts reach the backend over authenticated HTTP, and the submit command is the only component that pushes to the repository host, after confirming approval with the backend.

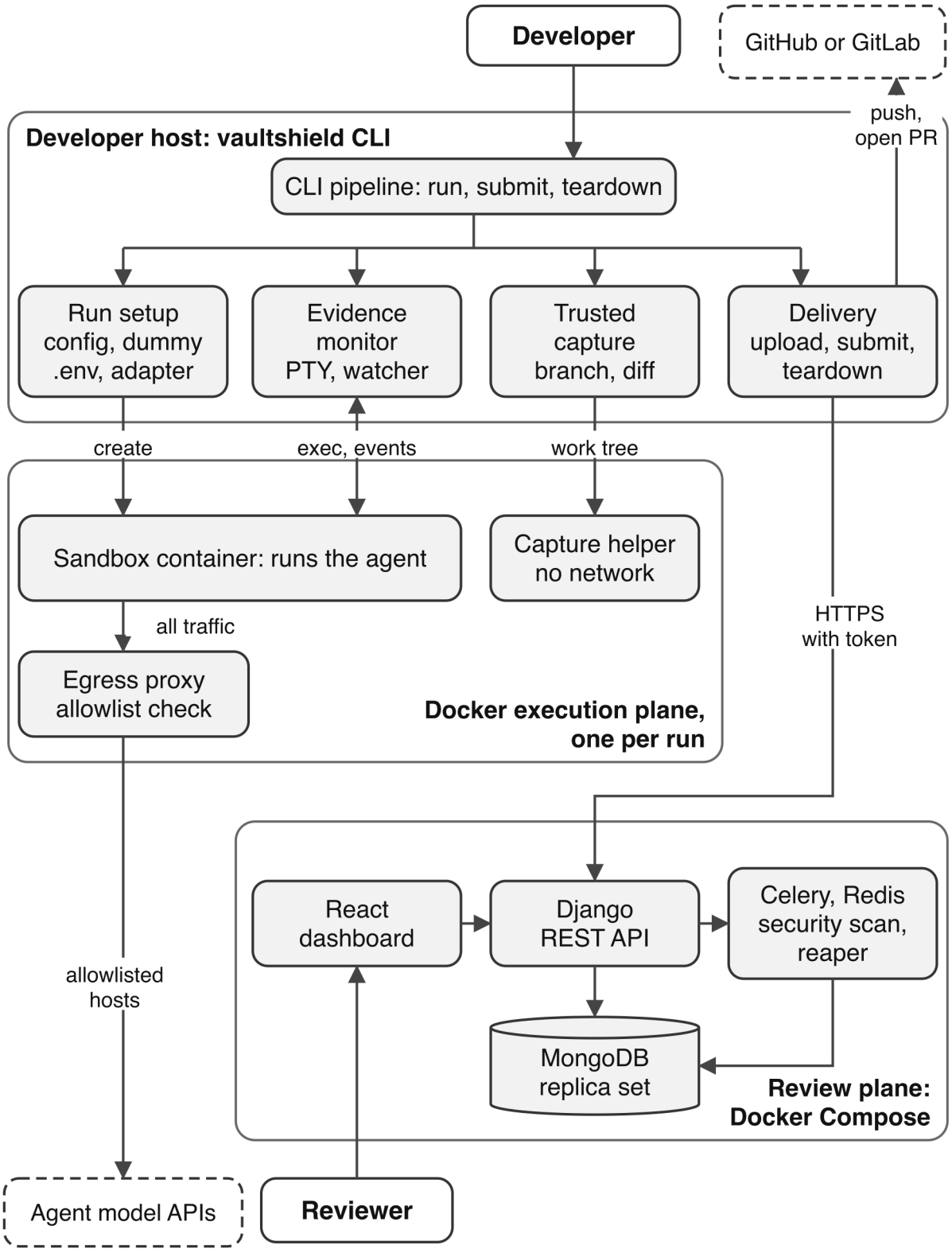



**Figure 3.5:** Overall system architecture

## 3.4 System Diagrams

System diagrams give a visual representation of the structure, behaviour and interactions of the proposed VaultShield system. The diagrams used are the UML Diagram, Activity Diagram, Use Case Diagram and Sequence Diagram.

### 3.4.1 UML Diagram

Figure 3.6 shows the persistent backend entities and the principal CLI-side classes. Run is the hub of the schema: RunEvent, Screenshot and SecurityFinding are many-per-run, while RunDiff and Approval are one-to-one, which makes "a run has exactly one change set and at most one decision" a schema-level guarantee. No backend field can store a repository-host token. On the CLI side, an AgentAdapter describes how its agent runs inside the Sandbox, the EgressGuard isolates the Sandbox's network, monitoring produces the RunEvent records, and the TrustedCapture helper produces the file entries stored in RunDiff.

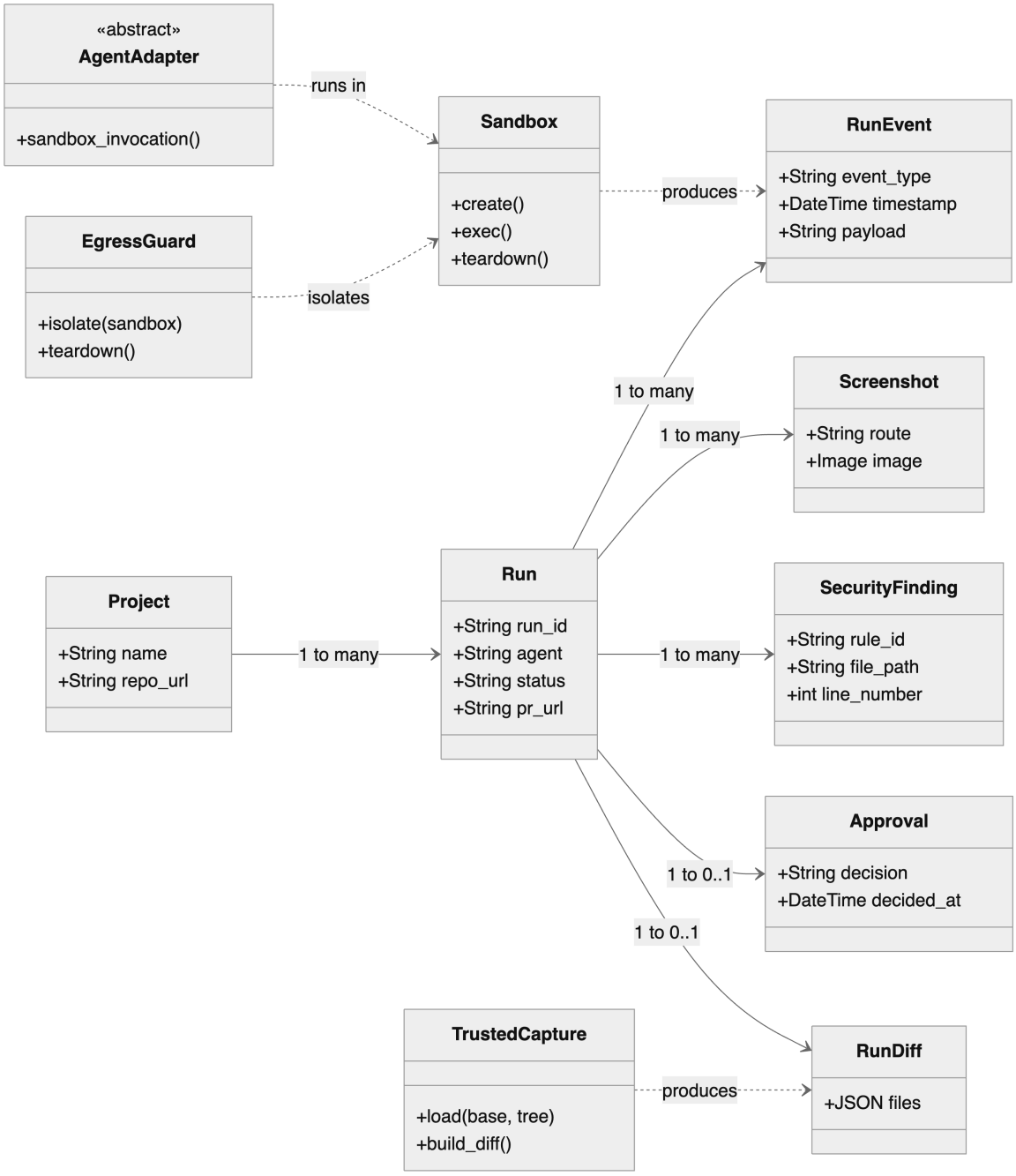



**Figure 3.6:** UML class diagram of the core backend entities and CLI classes

### 3.4.2 Activity Diagram

Figure 3.7 traces a run from setup through review and submission, reading each row in the direction of its arrows. The run is registered before the sandbox is created, policy failures are recorded, teardown runs on every exit path, and reviewer approval is the only route to submission.

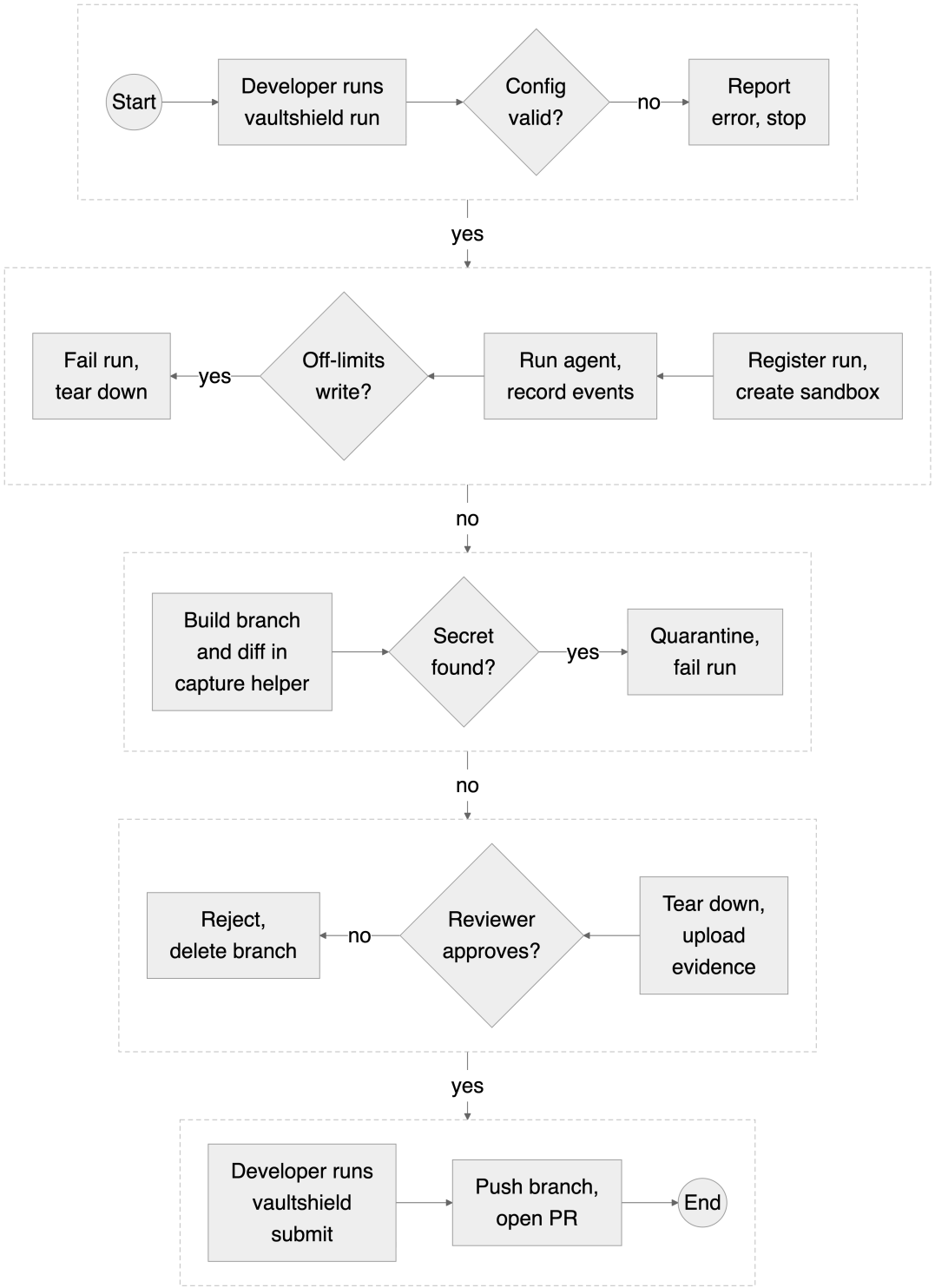



**Figure 3.7:** Activity diagram of a VaultShield run, from agent kickoff to PR creation

### 3.4.3 Use Case Diagram

Figure 3.8 shows the VaultShield actors and their interactions. The Developer configures and executes sandboxed runs, then submits or tears them down. The Reviewer inspects run evidence and approves or rejects the run. The System actor represents scanning and cleanup by Celery workers, and the Repository host receives the pushed branch and pull request. Executing a run includes dummy-credential generation, monitoring, change capture and upload, and may be extended by egress enforcement and screenshot capture.

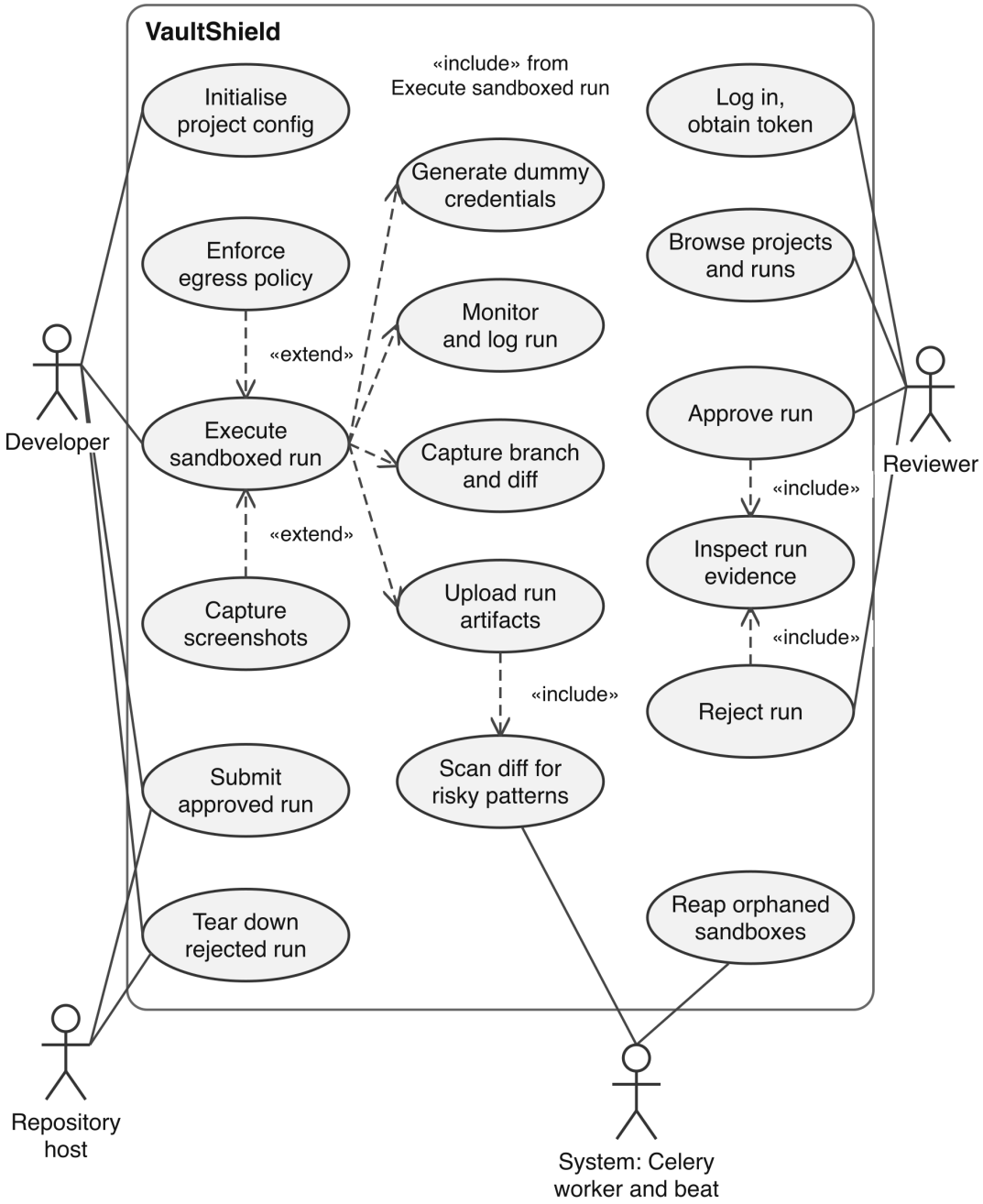



**Figure 3.8:** Use case diagram

### 3.4.4 Sequence Diagram

Figure 3.9 shows the message exchange for one approved run. Messages 1 to 9 cover the run, 10 and 11 the review, and 12 to 16 the submission. The backend learns of the run before Docker creates anything, the branch and diff are produced by a different container from the one the agent controlled, and submit re-reads the status immediately before pushing.

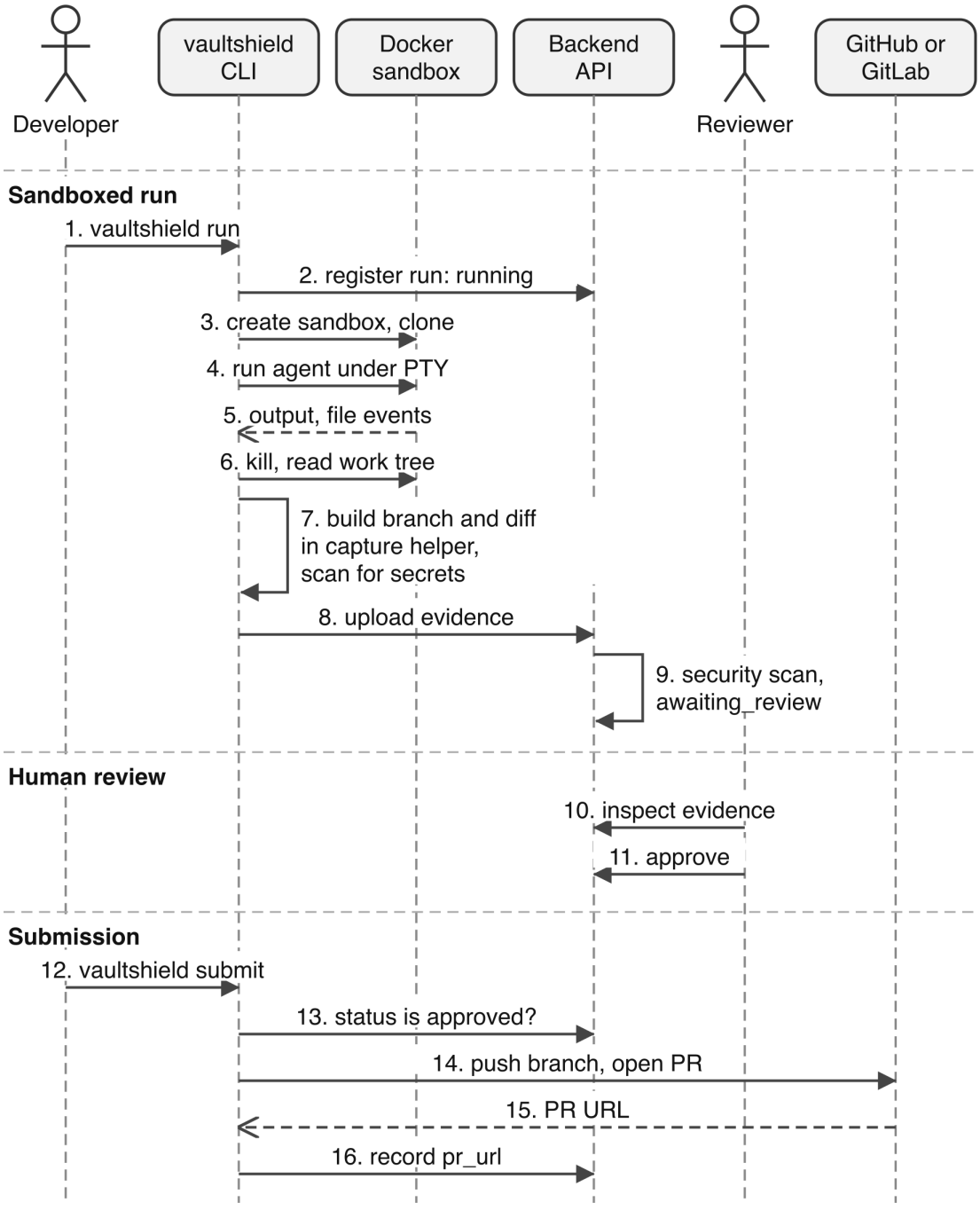



**Figure 3.9:** Sequence diagram of one complete approved run

## 3.5 Proposed Agentic AI Orchestrator (Objective 2, Semester VIII)

The Agentic AI Orchestrator is planned for Semester VIII and is not part of the Semester VII system evaluated in this report. This section specifies how it is intended to work so that its design can be reviewed before implementation.

### 3.5.1 Purpose

Coding agents often receive a one-line prompt such as "add a login page" and must guess the framework, the files to change, the constraints and what counts as done. Wrong guesses produce changes that the reviewer rejects, and they invite hallucinated files, APIs and dependencies. The Orchestrator runs before the agent starts. It reads the project's context, asks the developer a few clarifying questions, and turns the answers into one structured prompt that the agent then receives inside the existing sandbox. It does not replace the agent and it never executes code itself.

### 3.5.2 Proposed Workflow

Figure 3.10 shows the proposed workflow in three stages.

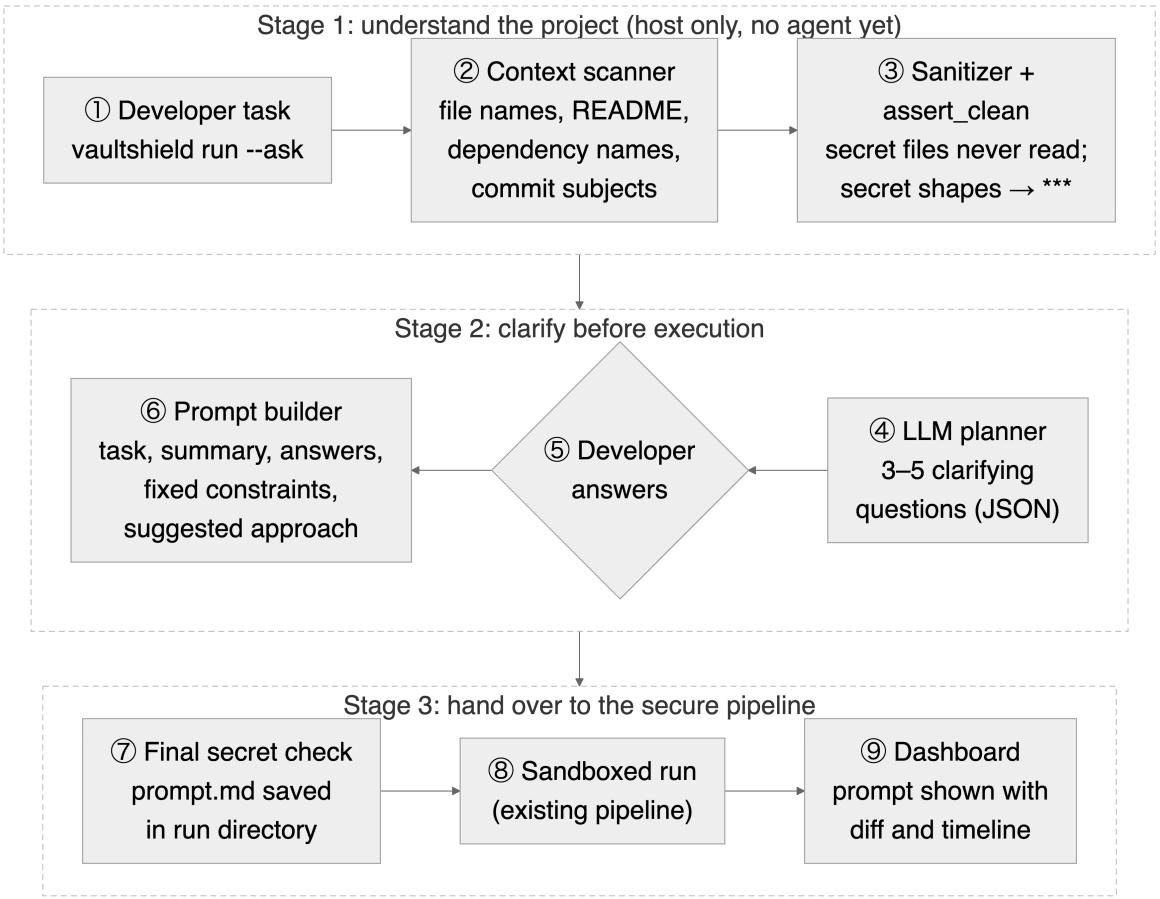



**Figure 3.10:** Proposed workflow of the Agentic AI Orchestrator (Semester VIII)

1. **Task entry.** The developer starts a run with a task description instead of a literal prompt, for example vaultshield run --ask “Add a login page”.
2. **Context scanning.** On the host, before any container exists, a scanner collects a bounded project summary: the directory listing to three levels, the start command and port detected by vaultshield init, dependency names from the package manifests, the beginning of the README and recent commit subjects.
3. **Sanitisation.** Every piece of text passes through a sanitiser that replaces known protected values and secret-shaped strings (API keys, private keys, tokens, passwords in URLs) with ***, followed by an assertion that rejects the request if anything secret-like remains. Secret files such as .env, key files and credential stores are never opened.
4. **Clarifying questions.** A language model receives the sanitised summary and the task and returns three to five clarifying questions in a strict JSON format, for example which authentication method to use or which files must not change. An invalid reply is retried once, and a fixed set of generic questions is used if the model fails.
5. **Developer answers.** The developer answers at the terminal, from a JSON file for scripted use, or skips the questions.
6. **Prompt construction.** The Orchestrator writes an enriched prompt with five parts: Task, Project summary, Answers, Constraints and Suggested approach. The Constraints section is fixed text that the model cannot change, and it tells the agent to refer to secrets by variable name only.
7. **Final check and hand-over.** The finished prompt is checked for secrets again, saved as prompt.md in the run directory and passed to the unchanged sandboxed pipeline described in Section 3.2.
8. **Review.** The dashboard shows the prompt next to the diff and timeline, so the reviewer can judge the change against what was actually asked.

### 3.5.3 Proposed Components and Expected Functionality

Table 3.2 lists the planned components and the function each is expected to provide.

**Table 3.2:** Planned components of the Agentic AI Orchestrator

| Component | Expected functionality |
|---|---|
| Context scanner | Reads a fixed list of non-secret files and produces a size-bounded project summary; refuses symbolic links so a README cannot point at a secret file |
| Sanitiser and leak check | Replaces protected values and secret-shaped text with ***, then blocks the model request if anything secret-like remains |
| Model provider | Sends the sanitised request to a configurable language model with the API key in a header, a 30-second timeout and provider-side storage disabled where the API allows it |
| Question generator | Produces three to five validated clarifying questions, with one retry and a generic fallback |
| Prompt builder | Assembles the five-part enriched prompt with a fixed Constraints section |

| Component | Expected functionality |
|---|---|
| Call log | Appends one metadata line per model call (time, provider, model, request size, outcome) to orchestrator.jsonl, with no request or reply content |
| Pipeline integration | Starts the existing sandboxed run with the enriched prompt and stores prompt.md as part of the run evidence |

### 3.5.4 Security Constraints and Evaluation Plan

The Orchestrator sends project context to an external model, so it is itself a new data-leak path and it must not weaken the guarantees of the Semester VII pipeline. Three constraints follow. First, no secret file is read and every outgoing request passes the leak check, so a failure blocks the request instead of sending it. Second, the Orchestrator runs before the run is registered and outside the sandbox, but it gains no new privileges: the agent still receives only dummy credentials and its change still goes through trusted capture and human approval. Third, a README containing hostile instructions could steer the model's questions or suggested approach, so the generated prompt is shown to the developer and stored as evidence for the reviewer rather than trusted blindly. Sanitisation by pattern is a heuristic; a secret in an unusual shape inside README text or a commit message may not be recognised, and this residual risk will be stated in the Semester VIII evaluation.

The Semester VIII evaluation will compare runs of the same real agents on the same tasks with a plain prompt and with an Orchestrator prompt, measuring the task success rate against each repository's tests, the reviewer approval rate, the number of hallucinated files, functions or packages in the diff, and the extra time and model cost the Orchestrator adds per run. A separate leak test will seed repositories with known secrets in README files, commit messages and file names and count how many reach the model request.

## 3.6 Threat Model

The threat model states what VaultShield protects, from whom, where the trust boundaries lie and which attacks it addresses. It follows the STRIDE categories (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service and Elevation of privilege).

### 3.6.1 Assets, Adversary and Assumptions

The protected assets are the developer's real secrets (the application .env, SSH keys, cloud credentials and the repository-host token), the developer's working tree and host system, the shared repository and its protected branches, the integrity of the evidence shown to the reviewer, and the agent's own model credential.

The adversary is the AI coding agent and anything that can influence it. The agent may simply be wrong, it may be steered by indirect prompt injection planted in the repository's README, issues or dependencies [7], or it may be deliberately hostile, for example a compromised agent binary or a malicious package that runs during dependency installation. The adversary controls every process inside the sandbox and can read and write every file there.

The following are assumed trusted: the developer's host operating system and Docker daemon, the VaultShield CLI and its capture helper image, the backend operator, and the repository host. In the single-user Semester VII system, the developer and the reviewer are the same trust domain.

### 3.6.2 Trust Boundaries

Figure 3.11 shows the three zones. The untrusted zone contains the agent and its inputs. The sandbox boundary is a per-run container that holds only a fresh clone, a read-only dummy credential file and, when egress control is enabled, a route to the filtering proxy. The trusted zone contains the CLI, the capture helper, the backend and the reviewer, and it is the only place real secrets exist. Labels T1 to T5 mark the main attack paths that cross a boundary.

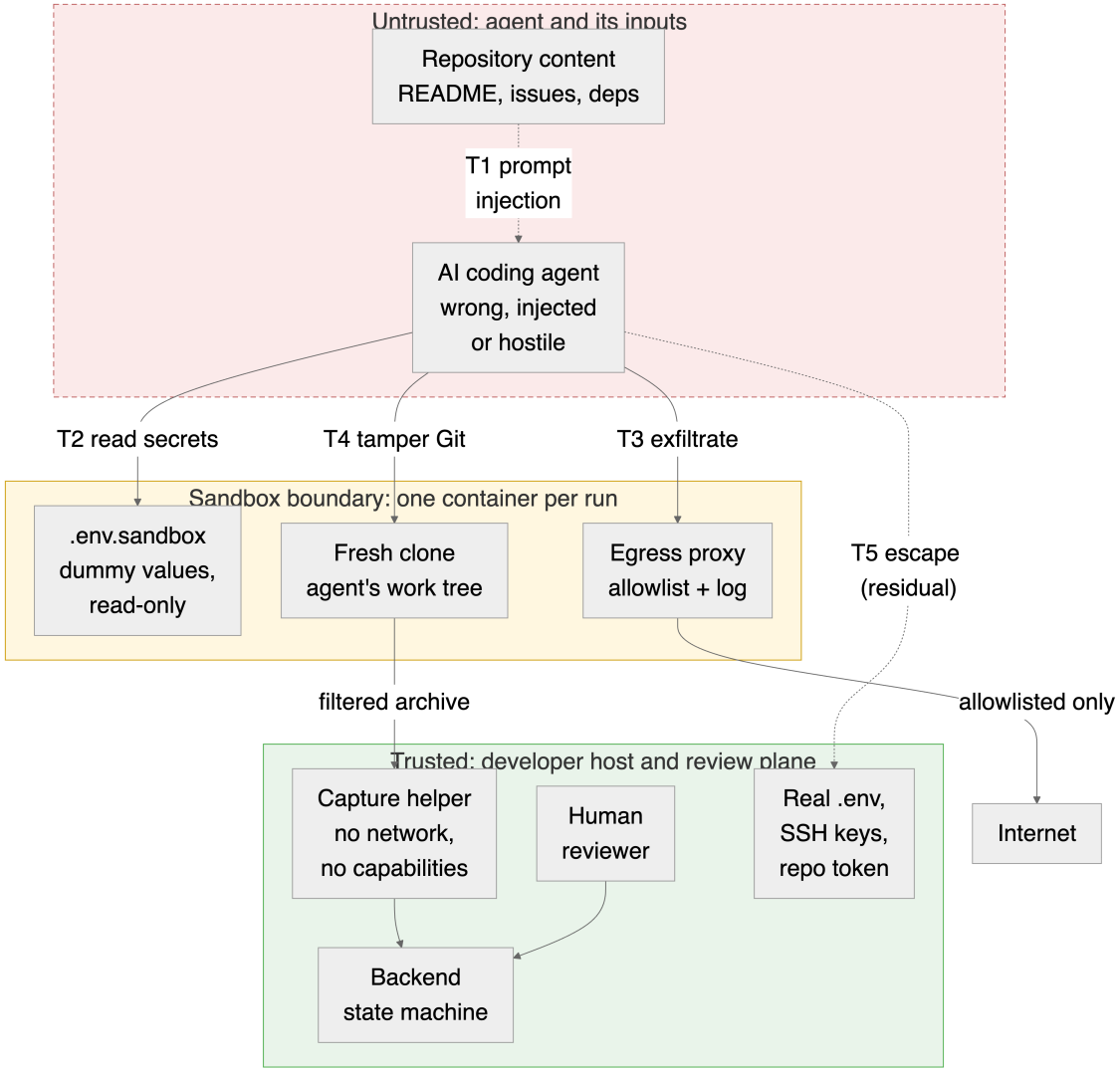



**Figure 3.11:** Threat model: trust zones and attack paths

### 3.6.3 Attacks and Mitigations

Table 3.3 lists the attacks considered, the VaultShield control for each, the evidence that the control works and the risk that remains.

**Table 3.3:** Threat model: attacks, mitigations and residual risk

| ID | Attack | STRIDE | VaultShield mitigation | Evidence | Residual risk |
|---|---|---|---|---|---|
| T1 | Prompt injection from repository content makes the agent act against the developer | T, I | Not prevented; its effects are contained by T2 to T9 and every change requires human approval | Design property | Hostile intent inside an approved change must be caught by the reviewer |
| T2 | Agent reads the real .env, SSH keys or home directory | I | Fresh clone inside the container; only a read-only dummy .env.sandbox is mounted; host paths never mounted | TC-01, TC-02, Fig. 7.8 | Values the developer pastes into the prompt or repository |
| T3 | Agent exfiltrates code or data over HTTP, raw TCP or DNS | I | Optional egress allowlist: internal network with no gateway, filtering proxy, public-address check, every decision logged | TC-03, 4 of 4 blocked | Egress is off unless enabled; uploads to an allowlisted host |
| T4 | Repository-host token or the agent's own model key leaks into the diff, logs or files | I | Token given only to the git clone process; token-hygiene scan fails the run and quarantines the diff | TC-06, 5 of 5 vectors | Deliberately encoded leaks |
| T5 | Agent tampers with Git state, hooks or configuration to hide or misreport its change | T, R | Trusted base recorded before the agent runs; separate capture helper with no network rebuilds the branch; unsafe Git configuration refused; hooks disabled | TC-05, 2 of 2 | None known for this path |
| T6 | Agent edits protected paths such as CI or infrastructure files | T | Off-limits paths recorded as violations and the run failed | TC-04 | Paths the developer did not list |

| ID | Attack | STRIDE | VaultShield mitigation | Evidence | Residual risk |
|---|---|---|---|---|---|
| T7 | Agent damages the host or leaves processes running | T, D | Agent runs only through exec in a disposable container; teardown on every exit path; scheduled reaper | Host unchanged 40 of 40; 0 leftovers | Resource exhaustion inside the sandbox (no default limits) |
| T8 | Container escape through the Docker socket, a kernel bug or a privileged mount | E | Docker socket never mounted; one file mounted; capture helper drops all capabilities | Probe, Fig. 7.8 | Shared kernel and root inside the default sandbox; addressed in Semester VIII |
| T9 | Unapproved change reaches the repository, or two reviewers race | E, R | Approval state machine in a MongoDB transaction; HTTP 409 for the loser; submit re-checks approval before any push | TC-07, TC-08, Fig. 7.7 | Status PATCH route and a single shared identity; role-based approval in Semester VIII |
| T10 | Evidence uploaded to the backend is altered or misattributed | T, S | Token-authenticated API; run identifier binds sandbox, branch and record; branch bundle has a SHA-256 | Backend tests | A holder of the API token can upload; no per-user signing |

The model makes the boundary of VaultShield's claims explicit. It contains a misbehaving agent and produces evidence the agent cannot falsify, but it does not judge intent: a malicious change that looks correct can still be approved by a careless reviewer. Container isolation, default-open egress and the single-user approval model are the three known weaknesses, and they map directly to the Semester VIII enhancements.

# Chapter 4

# Project Planning and Cost Estimation

## 4.1 Project Development Methodology

VaultShield was developed with an iterative and incremental methodology based on specification-driven development. The team first wrote a requirements specification in which every requirement carried numbered acceptance criteria, and a design document with the architecture, data model and numbered correctness properties. The design was split into small, testable tasks, and each short iteration implemented a slice of tasks together with unit and property-based tests and, where isolation was involved, a Docker-backed integration test. The major phases were:

1. **Problem Study and Feasibility Spike:** survey the literature and measure how a production agent handles .env secrets under four protection configurations.
2. **Requirements and System Design:** write numbered requirements and define the architecture, run lifecycle, data model, REST API and threat model.
3. **CLI Core and Evidence Capture:** implement configuration, credential shaping, the sandbox lifecycle, agent adapters, monitoring, screenshots and Git capture.
4. **Backend and Dashboard:** implement the models, REST API, authentication, security scan, scheduled tasks and the review dashboard.
5. **Submission and Hardening:** implement PR and MR submission and teardown, then add the egress proxy, the trusted capture helper and token hygiene.
6. **Migration and Protect Module:** move the backend to MongoDB with transactional approval, and add the Protect module.
7. **Evaluation and Documentation:** run the tests and performance benchmark and write this report.

## 4.2 Project Schedule and Gantt Chart

The Semester VII activities follow the project Gantt chart, reproduced as Figure 5.16, and the Semester VIII activities are planned extensions. Table 4.1 lists each activity with its duration and target period.

**Table 4.1:** Planned project schedule for VaultShield

| Sr. | Project activity | Duration | Target period |
|---|---|---|---|
| **Semester VII** | | | |
| 1 | Group formation and topic finalisation; identifying the scope and objectives of the project | 1 week | 10 Jul – 17 Jul 2026 |
| 2 | Discussing the project topic with the help of a paper prototype | 1 week | 17 Jul – 24 Jul 2026 |
| 3 | Identifying the functionalities of the project as per the objectives identified | 1 week | 24 Jul – 31 Jul 2026 |
| 4 | Literature review and research paper study | 1 week | 31 Jul – 7 Aug 2026 |
| 5 | Designing the graphical user interface | 3 weeks | 7 Aug – 28 Aug 2026 |
| 6 | Presentation I | 1 day | 4 Sep 2026 |
| 7 | Policy logic development | 1 week | 18 Sep – 25 Sep 2026 |
| 8 | Core model development and training | 1 week | 25 Sep – 1 Oct 2026 |
| 9 | Presentation II | 1 day | 9 Oct 2026 |
| 10 | Incorporating the suggestions from Presentation II and submitting a draft copy to the project guide for proofreading | 1 week | 16 Oct – 23 Oct 2026 |
| 11 | Draft submission to the guide for review | 2 days | 23 Oct – 24 Oct 2026 |
| 12 | Final black book submission | 1 week | 26 Oct – 30 Oct 2026 |
| 13 | External presentation and viva voce | 1 day | 2 Nov 2026 |
| **Semester VIII** | | | |
| 14 | Evaluation with real agents on sample repositories | 3 weeks | January 2027 |
| 15 | Default sandbox hardening and fixes for issues found in testing | 3 weeks | January – February 2027 |
| 16 | Verification of the Codex and Kiro egress allowlists; reviewer-identity separation study | 2 weeks | February 2027 |
| 17 | Usability study with reviewers and dashboard refinements | 2 weeks | March 2027 |
| 18 | Research paper drafting and submission | 2 weeks | March 2027 |
| 19 | Final report, demonstration and viva preparation | 3 weeks | March – April 2027 |

## 4.3 Team Roles and Responsibilities

The team of four divided responsibilities by architectural plane, with shared ownership of testing and documentation, as recorded in Table 4.2.

**Table 4.2:** Team roles and responsibilities

| Member | Primary role | Responsibilities |
|---|---|---|
| Aayush Subhash Balip | Backend and data engineering | Django REST API, MongoDB models, run state machine, artifact ingest, Celery tasks, Docker Compose stack |
| Krish Mahesh Patil | Frontend and user experience | React dashboard, run detail tabs, decision controls, accessibility and responsive layout |
| Muhammad Anis Momin | Security research and sandbox engineering | Sandbox lifecycle, credential shaping, egress proxy, trusted capture, token hygiene, threat model |
| Sameer Hasan Saiyed | Integration, testing and documentation | CLI pipeline, agent adapters, submit and teardown, test strategy, benchmark and report |

## 4.4 Software Size Estimation

### 4.4.1 KLOC-Based Estimation

The size of the delivered software was measured directly from the repository as logical source lines, excluding blank lines, comments, docstrings, migrations, lock files and generated content. Table 4.3 reports the counts.

**Table 4.3:** Measured software size by component

| Component | Product code, SLOC | Test code, SLOC |
|---|---:|---:|
| CLI package and tools | 12,404 | 16,697 |
| Backend | 1,150 | 2,552 |
| Frontend | 4,356 | 2,511 |
| Deployment configuration | 160 | – |
| **Total delivered product** | **18,070** | **21,760** |

The delivered product therefore measures KLOC = 18.07. Test code is excluded from the primary estimate, because COCOMO sizes delivered source instructions.

## 4.5 Effort Estimation Using COCOMO

### 4.5.1 Basic COCOMO Model

The Basic Constructive Cost Model [15] estimates effort and schedule from size alone. VaultShield was classified as an organic project, built by a small team in a familiar environment with flexible requirements, so the coefficients are a = 2.4, b = 1.05, c = 2.5 and d = 0.38. E is effort in person-months, T is development time in months and P is the average number of persons.

$E = a \times (KLOC)^{b} = 2.4 \times (KLOC)^{1.05}$
$T = c \times (E)^{d} = 2.5 \times (E)^{0.38}$
$P = E / T$

### 4.5.2 Calculation

With KLOC = 18.07, $(18.07)^{1.05} = 20.884$, so $E = 2.4 \times 20.884 = 50.12$ person-months. Then $(50.12)^{0.38} = 4.426$, so $T = 2.5 \times 4.426 = 11.06$ months, and $P = 50.12 / 11.06 = 4.53$ persons. Table 4.4 summarises the estimate.

**Table 4.4:** Basic COCOMO estimate

| Quantity | Formula | Value for KLOC = 18.07 |
|---|---|---:|
| Effort E | $2.4 \times KLOC^{1.05}$ | 50.12 person-months |
| Development time T | $2.5 \times E^{0.38}$ | 11.06 months |
| Average staffing P | E / T | 4.53 persons |

The estimate of about 4.5 persons over about 11 months agrees closely with the actual four-member team working across two semesters, which supports the organic classification.

## 4.6 Human Resource Cost

The project was completed by the four team members as part of their academic programme, and no salary, stipend or consultant fee was paid. Table 4.5 therefore records actual cash expenditure rather than a hypothetical value for unpaid effort.

**Table 4.5:** Actual human-resource cash expenditure

| **Cost item** | **Actual basis** | **Actual cost in ₹** |
|---|---|---:|
| Student development effort | Unpaid academic project work | 0 |
| Guide supervision and review | No project-specific payment | 0 |
| **Total** | | **0** |

## 4.7 Hardware, Software and Cloud Infrastructure Cost

The work used existing team laptops, open-source software and local infrastructure, and no paid cloud environment was commissioned, as shown in Table 4.6.

**Table 4.6:** Actual hardware, software and infrastructure cash expenditure

| **Category** | **Actual basis** | **Cost in ₹** |
|---|---|---|
| Hardware | Existing team laptops; no new equipment | 0 |
| Software | Python, Django, React, Git, Docker and test frameworks, all open source | 0 |
| Database and hosting | Local MongoDB replica set and free-plan repository hosting | 0 |
| Cloud and model APIs | No paid deployment or project-funded usage charge | 0 |
| **Total** | | **0** |

## 4.8 Testing, Deployment and Maintenance Cost

Testing and the MVP deployment ran on local machines with Docker, so no testing or deployment cost was incurred, as shown in Table 4.7. Future maintenance is not included because it has not been incurred.

**Table 4.7:** Actual testing, deployment and maintenance cash expenditure

| **Item** | **Actual basis** | **Actual cost in ₹** |
|---|---|---|
| Automated and Docker-backed tests | Local machines and open-source frameworks | 0 |
| MVP deployment | Local Docker Compose stack | 0 |
| **Total** | | **0** |

## 4.9 Total Estimated Project Cost

Table 4.8 consolidates the actual project-specific cash expenditure.

**Table 4.8:** Total actual project cash expenditure

| **Cost head** | **Source** | **Actual cost in ₹** |
|---|---|---|
| Human resources | Table 4.5 | 0 |
| Hardware, software and infrastructure | Table 4.6 | 0 |
| Testing, deployment and maintenance | Table 4.7 | 0 |
| **Total actual project cash expenditure** | | **0** |

## 4.10 Assumptions and Cost Justification

Only amounts paid specifically for the project are included. Unpaid student effort, depreciation of equipment already owned, normal utilities and possible future maintenance are not given hypothetical monetary values, and tools already available at no extra charge count as zero. The actual cash expenditure for VaultShield is therefore ₹0. This basis is

separate from the COCOMO estimate, which describes effort and schedule rather than money spent.

## 4.11 Risk Assessment and Mitigation

Table 4.9 lists the main technical, security and project risks with their likelihood, impact and mitigation.

**Table 4.9:** Risk assessment and mitigation

| Risk | Likelihood | Impact | Mitigation |
|---|---|---|---|
| Container escape through a kernel or Docker vulnerability | Low | Very high | Docker socket never mounted; only a dummy credential file is mounted; gVisor or microVM isolation planned |
| Malicious dependency running install hooks | Medium | High | Installs only inside the disposable sandbox with dummy credentials; egress allowlist available |
| Network exfiltration when egress control is off | Medium | High | Log-only and allowlist modes through a hardened proxy; default-deny egress planned |
| Real secrets reaching the sandbox or artifacts | Low | Very high | Dummy credentials, one-shot clone credential, token-hygiene scan with diff quarantine |
| Agent tampering with Git state to hide changes | Low | High | Trusted capture in a separate helper from a pre-agent base |
| Approval gate bypassed by a token holder | Medium | Medium | Single-developer trust domain documented; reviewer identities planned |
| Upstream agent CLI changes break an adapter | High | Medium | Adapter interface isolates changes; mock adapter keeps the pipeline testable |
| Schedule slippage or member unavailability | Medium | Medium | Working pipeline after every iteration; shared module knowledge through testing |

# Chapter 5

# Project Implementation

## 5.1 Code Snippets

### 5.1.1 Launching the Sandbox Container

The first snippet is the core of Sandbox.create. Before any Docker call it checks that the credential file exists, because Docker silently creates an empty directory at a missing bind-mount source. Egress options are passed only when they are set, and any failure removes the partial container and raises a single SandboxError. As Figure 5.1 shows, the only host path given to the container is the read-only credential file, and teardown force-removes the container from a finally block so that it runs on every exit path.

Sandbox container creation and teardown

```
# Sandbox.create (body, docstring elided)
try:
    if not env_file.is_file():
        raise SandboxError(
            f"Sandbox creation failed for run {self.run_id}: "
            f"credential file not found at {env_file}"
        )

    self._ensure_image(base_image)

    egress_kwargs = {
        key: value
        for key, value in (
            ("network", network),
            ("cap_drop", cap_drop),
            ("sysctls", sysctls),
            ("extra_hosts", extra_hosts),
        )
        if value is not None
    }
    self.container = self._client.containers.create(
        base_image,
        command=_KEEP_ALIVE_CMD,                # ["tail", "-f", "/dev/null"]
        name=container_name(self.run_id),        # "vaultshield-run-<run_id>"
        working_dir=_WORKDIR,
        volumes={
            str(env_file.resolve()): {
                "bind": _ENV_MOUNT_PATH,         # /workspace/.env.sandbox
                "mode": "ro",
            }
        },
        ports=(
            {f"{publish_port}/tcp": None} if publish_port is not None else None
        ),
        **egress_kwargs,
    )
    self.container.start()

    self._clone_repo(repo_url, resolve_credential=resolve_credential)
    self._install_dependencies(project_dir)
except Exception as exc:
    self._cleanup_after_failure()
    if isinstance(exc, SandboxError):
        raise
    raise SandboxError(
        f"Sandbox creation failed for run {self.run_id}: {exc}"
    ) from exc

# Sandbox.teardown (docstring elided): idempotent, kills every exec'd process
def teardown(self) -> None:
    if self.container is None:
        return
    try:
        self.container.remove(v=True, force=True)
    except docker.errors.NotFound:
        pass
    finally:
        self.container = None
```

**Figure 5.1:** Sandbox container creation and teardown

### 5.1.2 One-Shot Credential for Private Clones

The second snippet clones a private repository without giving the agent the repository-host token. The token is placed only in the environment of the single git clone exec, and the command line names environment variables rather than containing their values. As Figure 5.2 shows, the token is therefore absent from the remote URL, the process arguments, the container environment and error messages, and its lifetime is exactly the duration of the clone.

Authenticated clone with a one-shot credential helper

```
# Sandbox._clone_repo (docstring elided)
provider = _clone_credential_provider(repo_url)      # strict github/gitlab
allowlist
credential, unavailable_reason = self._clone_credential(
    provider, resolve_credential
)

if credential is not None:
    from vaultshield.submit import git_credential_args

    assert provider is not None  # implied by credential is not None
    config_args, env = git_credential_args(credential, provider)
    self._exec_or_raise(
        # `config_args` names the two environment variables the helper
        # reads; the token itself is only ever in `env`.
        ["git", *config_args, "clone", repo_url, _REPO_DIR],
        workdir=_WORKDIR,
        step="git clone",
        environment=env,                 # applies to this one exec only
        secrets=(credential.token,),     # redacted from any error message
    )
    return

try:
    self._exec_or_raise(
        ["git", "clone", repo_url, _REPO_DIR],
        workdir=_WORKDIR,
        step="git clone",
        environment={_GIT_TERMINAL_PROMPT_ENVVAR: "0"},
    )
except SandboxError as exc:
    if unavailable_reason is None:
        raise
    raise SandboxError(
        f"Sandbox creation failed for run {self.run_id} during git "
        f"clone: {repo_url} could not be cloned, and no usable "
        f"{provider} credential was available to authenticate it. If "
        f"the repository is private, {unavailable_reason} "
        f"Original git failure: {exc}"
    ) from exc
```

**Figure 5.2:** Authenticated clone with a one-shot credential helper

### 5.1.3 Event Logging and Off-Limits Enforcement

The third snippet, in Figure 5.3, shows the event record type, the off-limits path matcher and the append-only JSON Lines writer. Every record has exactly three fields matching the backend's RunEvent model, and json.dumps keeps each record on one line, so a partially written log stays readable. The monitor raises OffLimitsViolation only after the writer has run.

Event records, off-limits matching and the JSON Lines writer

```
@dataclass(frozen=True)
class EventLogRecord:
    event_type: Literal["output", "file_change", "violation"]
    timestamp: str  # ISO-8601
    payload: str


def _match_off_limits(
    changed_path: str,
    workdir: Path,
    entries: tuple[str, ...],
) -> str | None:
    # (docstring elided) returns the first violated entry, or None
    if not entries:
        return None
    candidates = _path_candidates(changed_path, workdir)
    for entry in entries:
        for candidate in candidates:
            if candidate == entry or candidate.startswith(entry + "/"):
                return entry
            if fnmatch.fnmatch(candidate, entry):
                return entry
    return None


def _append_event_log(path: Path, records: Iterable[EventLogRecord]) -> Path:
    # (docstring elided) append-only: an existing log is never rewritten
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("a", encoding="utf-8", newline="\n") as log_file:
        for record in records:
            log_file.write(json.dumps(asdict(record), ensure_ascii=False) + "\n")
    return path
```

**Figure 5.3:** Event records, off-limits matching and the JSON Lines writer

### 5.1.4 Egress Allow/Deny Decision

The fourth snippet, in Figure 5.4, is the egress proxy's decision function, evaluated for every CONNECT request and TLS connection. A host that is not on the allowlist is never resolved in allowlist mode, IP literals must match an IP entry exactly, and every resolved address must be publicly routable, so an allowlisted name cannot be pointed at an internal address. In log-only mode only the allowlist rule is relaxed.

Egress decision for one request target

```
def decide(
    entries: Sequence[AllowEntry],
    mode: str,
    host: str,
    port: int,
    resolver: Resolver = default_resolver,
    *,
    address_policy: Callable[[str], bool] = is_public_address,
) -> Decision:
    norm = normalise_host(host)
    if norm is None or not isinstance(port, int) or not 1 <= port <= 65535:
        return Decision("deny", "bad_request", host if norm is None else norm,
                        port if isinstance(port, int) else 0)

    if _ip_or_none(norm) is not None:
        if not host_listed(entries, norm):
            return Decision("deny", "ip_literal", norm, port)
        if not matches(entries, norm, port):
            return Decision("deny", "port_not_allowed", norm, port)
        if not address_policy(norm):
            return Decision("deny", "private_address", norm, port)
        return Decision("allow", None, norm, port, (norm,))

    verdict = "allow"
    reason: str | None = None
    if not host_listed(entries, norm):
        if mode != "log-only":
            return Decision("deny", "not_in_allowlist", norm, port)
        verdict, reason = "would_deny", "not_in_allowlist"
    elif not matches(entries, norm, port):
        return Decision("deny", "port_not_allowed", norm, port)

    addresses = _resolve_with_timeout(resolver, norm, port)
    if not addresses:
        return Decision(verdict, reason, norm, port, ())
    if not all(address_policy(address) for address in addresses):
        return Decision("deny", "private_address", norm, port)
    ordered = sorted(addresses, key=lambda a: 0 if ":" not in a else 1)
    return Decision(verdict, reason, norm, port, tuple(dict.fromkeys(ordered)))
```

**Figure 5.4:** Egress decision for one request target

### 5.1.5 Branch Commit and Structured Diff

The fifth snippet, in Figure 5.5, runs inside the trusted capture helper. commit_run_branch refuses unsafe repository configuration and creates exactly one commit, using --allow-empty so that a run which changed nothing still produces a reviewable result and --no-verify so that no repository hook runs. build_structured_diff disables rename detection, external diff programs and text conversion filters.

Committing the run branch and computing the diff

```
def commit_run_branch(sandbox: Sandbox, run_id: str) -> str:
    branch = run_branch_name(run_id)              # "vaultshield/run-<run_id>"

    # Refuse a gitfile/symlinked `.git` and any filter/diff driver
    # BEFORE `git add`, which is what would run a filter.
    _assert_capture_repo_is_safe(sandbox)

    _exec_or_raise(sandbox, _git("checkout", "-b", branch),
                   step=f"creating branch {branch}")
    _exec_or_raise(sandbox, _git("add", "--all"),
                   step="staging working-tree changes")
    _exec_or_raise(
        sandbox,
        _git("-c", f"user.name={COMMITTER_NAME}",
             "-c", f"user.email={COMMITTER_EMAIL}",
             "commit", "--allow-empty", "--no-verify",
             "-m", _commit_message(run_id)),
        step=f"committing changes to {branch}",
    )
    return branch


# build_structured_diff (excerpt): per-file counts, then size-capped patches
result = _exec_or_raise(
    sandbox,
    _git("diff", "--numstat", "-z", "--no-renames",
         "--no-ext-diff", "--no-textconv", base_ref, branch),
    step=f"diffing {base_ref}..{branch}",
)
records = _parse_numstat(_raw_stdout(result))
metadata_cost = _metadata_cost(records)
if metadata_cost > MAX_DIFF_REQUEST_BYTES:          # 2 MiB upload budget
    raise GitCaptureError(
        "Git capture failed while sizing diff.json: the entries for "
        f"{len(records)} changed files take {metadata_cost} bytes even "
        f"with every patch left out, over the {MAX_DIFF_REQUEST_BYTES}-byte "
        "upload cap (MAX_DIFF_REQUEST_BYTES), so the backend would reject "
        "the diff"
    )
```

**Figure 5.5:** Committing the run branch and computing the diff

### 5.1.6 Approval State Machine

The sixth snippet, in Figure 5.6, is the inner function of apply_decision. The status check and status write are a single conditional update, and the Approval record is created in the same MongoDB transaction, so only one of two concurrent decisions can match. The loser receives HTTP 409 naming the run's actual status; before the retry wrapper was added, 54 of 72 losing calls in a concurrency test received a server error instead.

Transactional approve/reject decision

```
ALLOWED_DECISIONS_FROM = frozenset({"awaiting_review"})

def apply_decision(run: Run, decision: Decision) -> Run:
    # (docstring elided)
    decided_at = to_mongo_precision(timezone.now())

    def record_decision() -> None:
        with transaction.atomic():          # django_mongodb_backend's atomic()
            recorded_decision = (
                Approval.objects.filter(run=run)
                .values_list("decision", flat=True).first()
            )
            if recorded_decision is not None:
                raise StateTransitionError(
                    _stored_status_of(run), recorded_decision=recorded_decision
                )

            transitioned = Run.objects.filter(
                pk=run.pk, status__in=ALLOWED_DECISIONS_FROM
            ).update(status=decision, updated_at=decided_at)

            if not transitioned:
                raise StateTransitionError(_stored_status_of(run))

            Approval.objects.create(run=run, decision=decision)

    # Retried on MongoDB's TransientTransactionError only. "Measured at 54
    # spurious errors out of 72 losing calls before this was added."
    retry_on_transient_transaction_error(record_decision)

    run.status = decision
    run.updated_at = decided_at
    return run
```

**Figure 5.6:** Transactional approve/reject decision

## 5.2 Steps to Access the System

The system requires Python 3.11 or later, a running Docker daemon, Git, Node.js 18 or later, a MongoDB replica set or Atlas cluster, and Playwright Chromium. The commands assume macOS or Linux.

1. Clone the repository and copy .env.example to .env, setting VAULTSHIELD_ MONGODB_URL to the MongoDB connection string.
2. Start the backend and dashboard with docker compose up –build. The API is served on http://localhost:8000 and the dashboard on http://localhost:5173.
3. Create a user with docker compose exec web python manage.py createsuperuser.
4. Install the CLI:
   cd cli && python3 -m venv .venv && source .venv/bin/activate && pip install -e ".[dev]" && playwright install chromium
5. Authenticate with vaultshield login –backend-url http://localhost:8000 –username <user>, create a project in the dashboard and note its identifier.
6. In a checkout of the target repository, run vaultshield init to write .vaultshield.yml.

7. Execute a sandboxed run, for example with the mock agent:
   vaultshield run –agent=mock –prompt=”add a file” –repo-url=”$REPO” –project ”$PROJECT_ID”
8. Open the dashboard, select the run, inspect the Diff, Timeline, Screenshots and Security tabs, and choose Approve or Reject.
9. For an approved run, run vaultshield submit <run-id> to open the pull or merge request; for a rejected run, run vaultshield teardown <run-id>.

## 5.3 System Implementation

The dashboard screenshots below were captured by the repository’s layout tool, which serves the built dashboard with fixture data, so they show the real interface with illustrative data.

### 5.3.1 Command-Line Interface

The CLI provides the pipeline commands init, login, run, submit and teardown and the supporting commands protect and exec. During vaultshield run, the terminal prints one line per pipeline milestone, as shown in Figure 5.7, and each run writes a run directory containing the dummy credential file, events.jsonl, diff.json, the branch bundle and the screenshots.

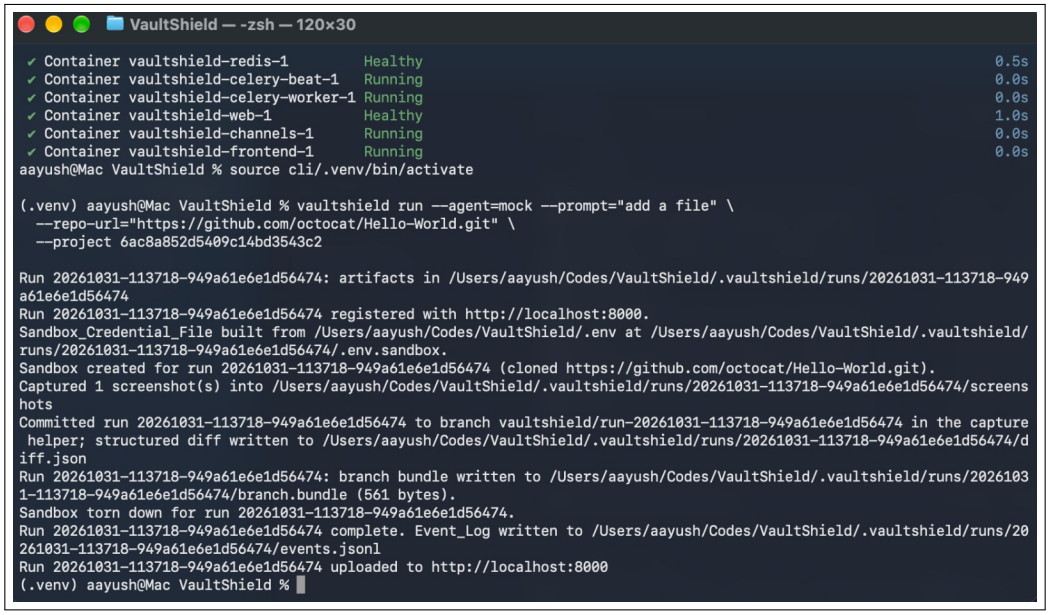



**Figure 5.7:** Terminal output of a successful vaultshield run with the mock agent

### 5.3.2 Landing Page and Projects Dashboard

The landing page in Figure 5.8 introduces the product with a one-line installer command and links to sign in. After sign-in, the projects overview in Figure 5.9 shows one card per project with its repository URL, a sidebar of projects and a New project button.

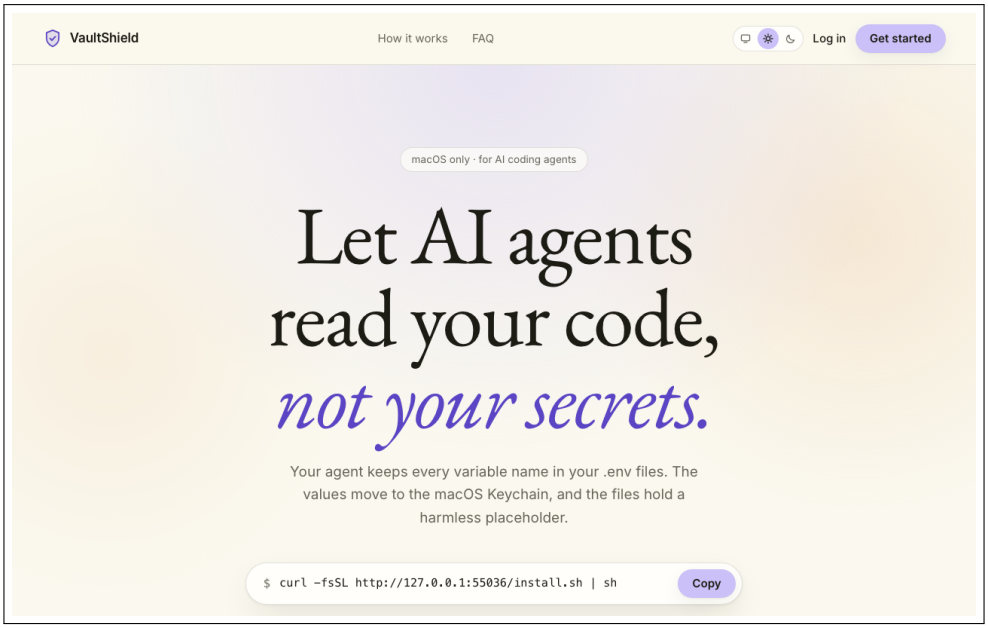



**Figure 5.8:** Landing page

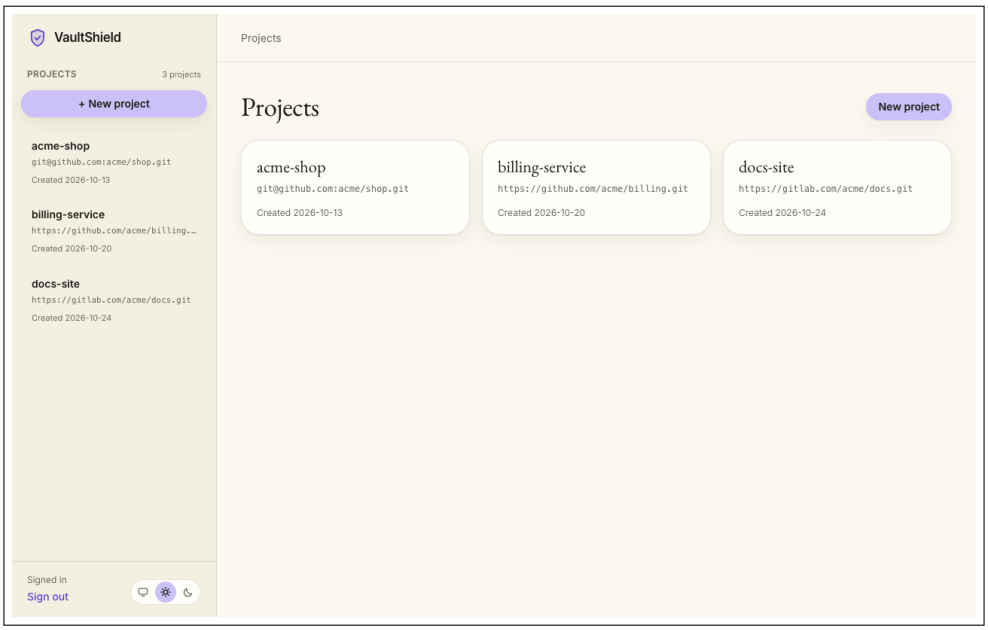



**Figure 5.9:** Projects dashboard

### 5.3.3 Run History

The run history in Figure 5.10 lists a project's runs newest first with the run identifier, agent, status badge and creation time, with status filters, search by run identifier and pagination.

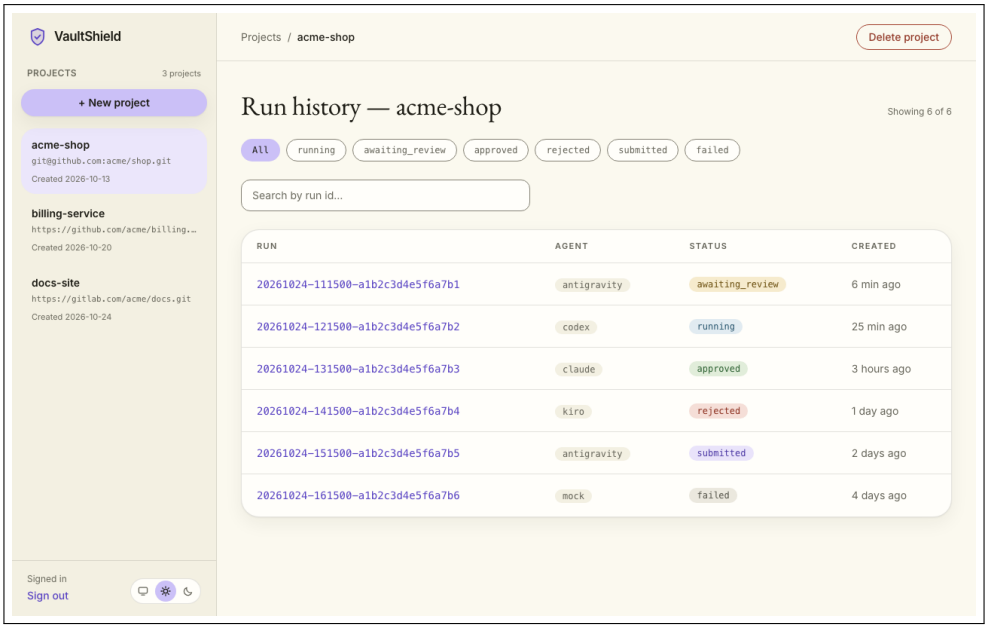



**Figure 5.10:** Run history with status filters

### 5.3.4 Run Detail Views

The run detail page has a header with the run's status, a decision banner while the run is awaiting review, and four tabs. The Diff tab in Figure 5.11 lists every changed file with its addition and deletion counts. The Timeline tab in Figure 5.12 shows the event log in order, giving the reviewer a view of what the agent did as well as what it changed [14]. The Screenshots tab in Figure 5.13 shows one image per configured route, and the Security tab in Figure 5.14 lists the findings of the automated scan with their rule, file and line. Findings inform the reviewer but do not block approval.

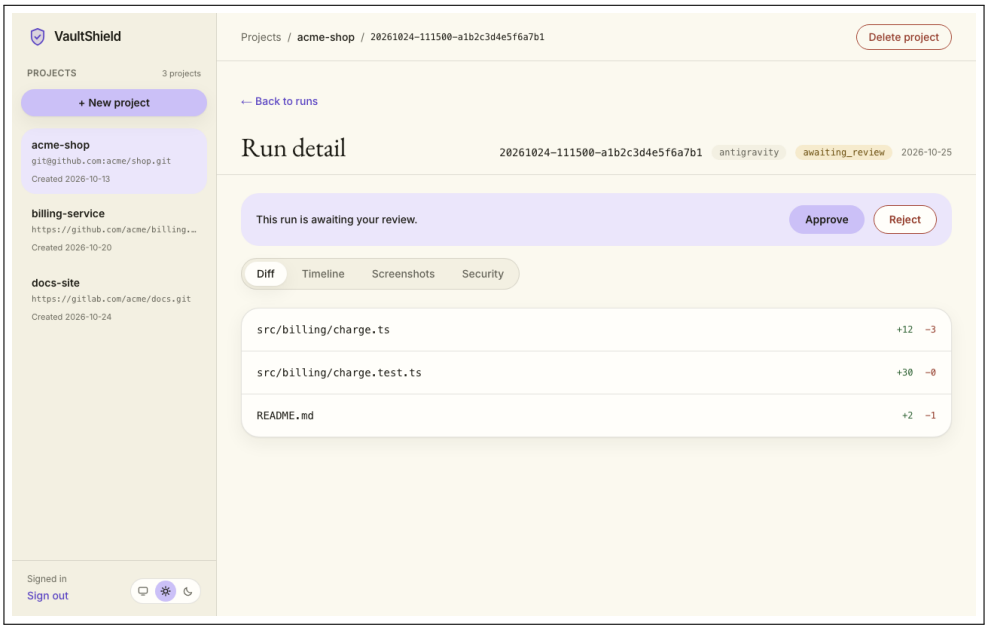



**Figure 5.11:** Run detail – Diff tab

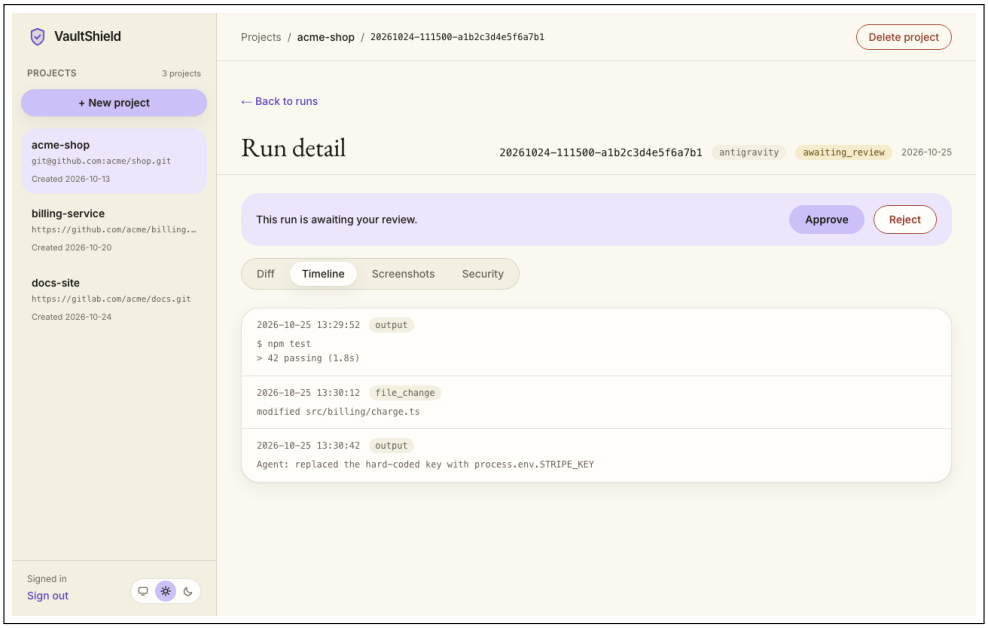



**Figure 5.12:** Run detail – Timeline tab

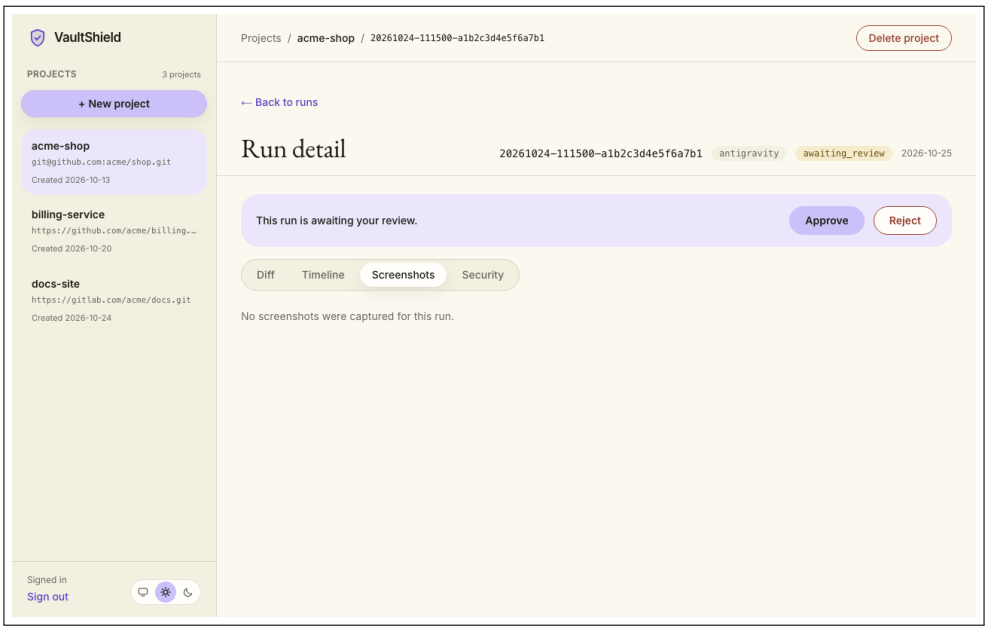



**Figure 5.13:** Run detail – Screenshots tab

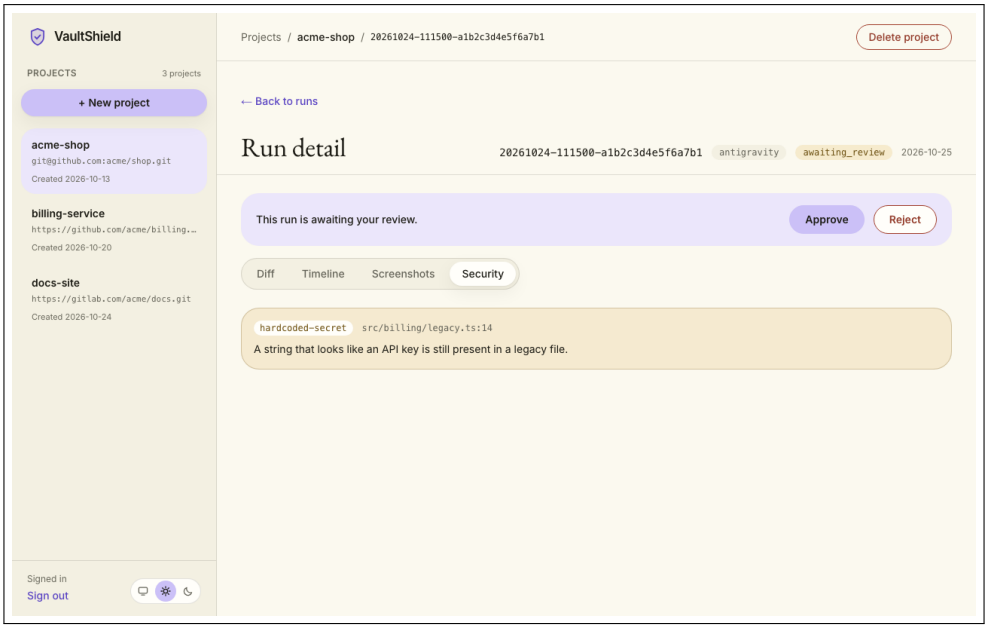



**Figure 5.14:** Run detail – Security tab

### 5.3.5 Approval Decision and Submission

When the reviewer approves a run, the buttons are replaced by the status “approved, awaiting submission” and the exact vaultshield submit command, as shown in Figure 5.15, because the backend never opens a pull request on its own.

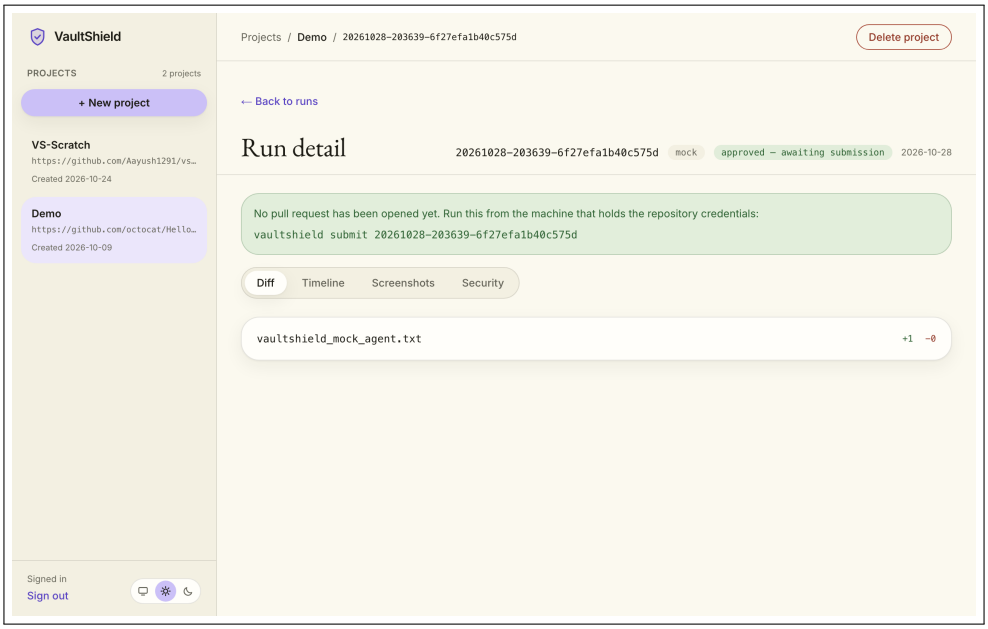



**Figure 5.15:** Run detail after approval, showing the submit command

### 5.3.6 Implementation Summary

The CLI creates the sandbox, monitors the agent, captures the change on a dedicated branch and uploads the evidence, while the dashboard presents that evidence and records the reviewer’s decision. Producing a change and accepting it therefore remain separate

steps.

## 5.4 Timeline Sem VII

The Gantt chart in Figure 5.16 shows the Semester VII activities across thirteen weeks, from group formation on 10 July 2026 to the external presentation and viva voce on 2 November 2026. The activities are grouped into four phases: project conception and initiation, design and implementation, evaluation and documentation, and final submission. All four members are owners of every activity, and every Semester VII activity is complete.

## 5.5 Semester VII Implementation and Semester VIII Plan

Everything described in this chapter, and evaluated in Chapters 6 and 7, was implemented in Semester VII. Table 5.1 lists the implemented features and Table 5.2 the features planned for Semester VIII, so that no planned item is mistaken for a delivered one.

**Table 5.1:** Features implemented in Semester VII

| Area | Implemented feature | Objective |
|---|---|---|
| CLI and adapters | init, login, run, submit and teardown commands; mock, codex, kiro and antigravity adapters behind one interface | 1 |
| Sandbox | One Docker container per run, fresh clone, read-only dummy .env.sandbox, one-shot clone token, guaranteed teardown | 3 |
| Egress control | Per-run internal network, hardened proxy, log-only and allowlist modes, per-agent allowlists | 3 |
| Monitoring | PTY output capture, polling in-container file watcher, off-limits paths, JSON Lines event log, screenshots | 4 |
| Trusted capture | Pre-agent trusted base, hardened capture helper, one-commit run branch, structured diff, verified bundle | 4 |
| Security analysis | Token-hygiene scan with diff quarantine; Celery static scan with five rules | 4 |
| Backend | Django REST API on MongoDB, run state machine, transactional approve and reject, sandbox reaper, stuck-approval flag | 5 |
| Dashboard | Projects, run history, run detail with Diff, Timeline, Screenshots and Security tabs, decision controls | 5 |
| Submission | Approval-gated pull or merge request to GitHub or GitLab; teardown of rejected branches | 5 |
| Protect module | Keychain-backed .env protection and vaultshield exec for agents run outside the sandbox | 3 |

**Table 5.2:** Enhancements planned for Semester VIII

| Enhancement | Planned work | Reason (Semester VII gap) |
|---|---|---|
| Agentic AI Orchestrator | Context scanner, sanitiser, clarifying questions, enriched prompt, integration with run (Section 3.5) | Objective 2 not implemented |

| Enhancement | Planned work | Reason (Semester VII gap) |
|---|---|---|
| Stronger sandbox isolation | Non-root user, dropped capabilities, no-new-privileges, CPU, memory and process limits, default-deny egress, gVisor or microVM runtime | Shared kernel; root in the default sandbox; egress opt-in |
| Improved monitoring | Event-driven file watcher, event-log cross-check against the diff, live streaming, inline patch view, fix for the sign-in read deadline | Watcher missed 8 of 40 writes; Section 6.6.2 |
| Role-based approval | Reviewer accounts, role-based access control, gated status transitions, decision attribution, multiple approvals | Single-user trust domain; status PATCH can set approved |

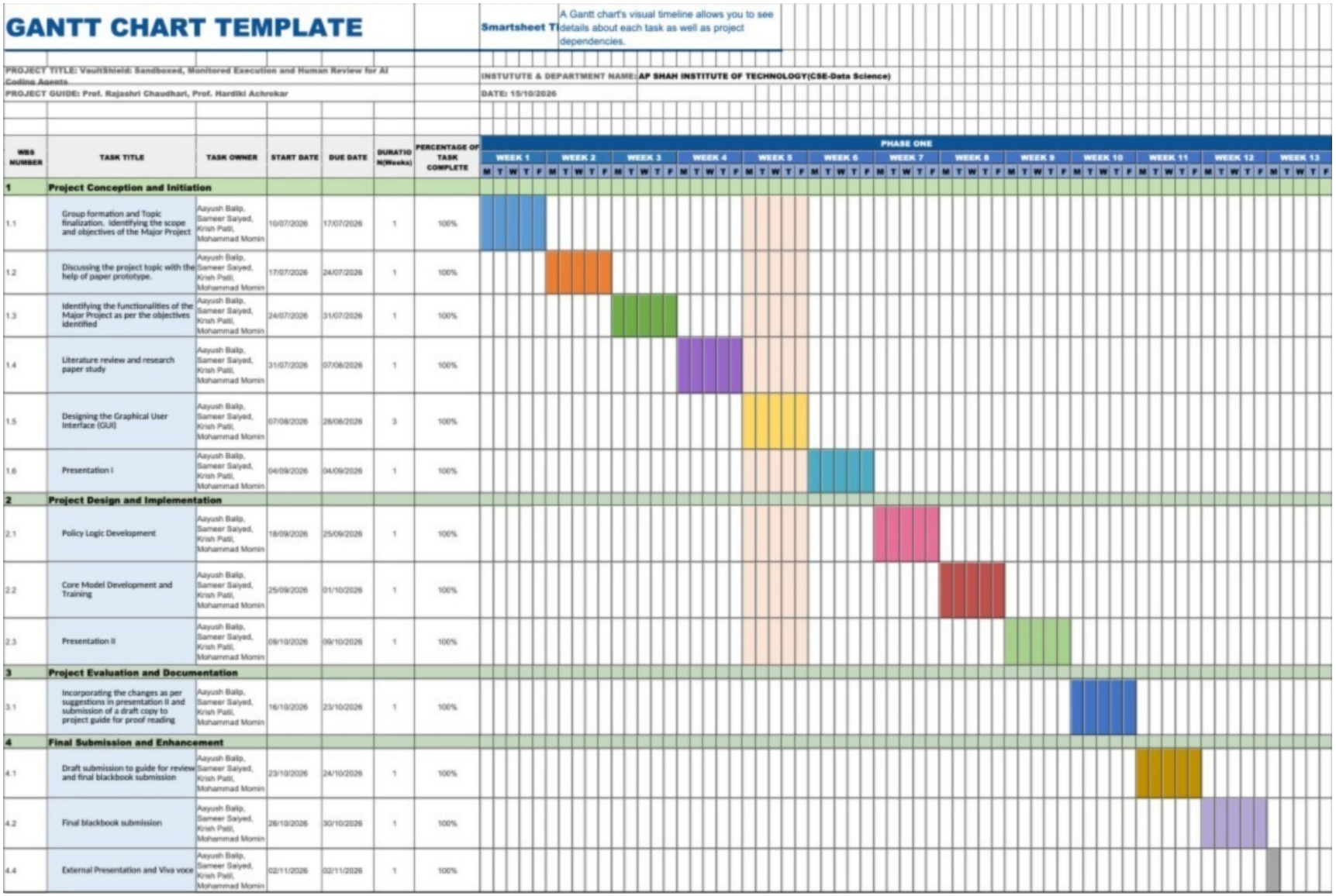



**Figure 5.16:** Gantt chart of the VaultShield major project

# Chapter 6

# Testing

## 6.1 Software Testing

VaultShield's value lies mostly in negative guarantees: the agent must not reach the host, real secrets must not enter the sandbox, a token must not appear in any artifact, and an unapproved run must not become a pull request. The team therefore traced every acceptance criterion to at least one test, expressed invariants as property-based tests with Hypothesis and fast-check, tested isolation claims against real Docker containers with a probing agent rather than mocks, and tested the backend against a real MongoDB replica set because the approval gate depends on genuine transaction semantics. Test fixtures redirect the home directory to an empty temporary directory so that no test can touch the developer's real files.

## 6.2 Types of Testing

Testing was organised into functional testing and system testing. Table 6.1 lists the test levels used within them.

**Table 6.1:** Types of testing performed

| Level | Scope | Tools |
|---|---|---|
| Unit | Configuration parsing, credential shaping, off-limits matching, diff parsing, egress decisions, React components | pytest, Hypothesis, Jest, fast-check |
| Integration | CLI with real Docker, backend with MongoDB, upload with the API | pytest with Docker and a replica set |
| Security-focused | Isolation, secret suppression, network blocking, tamper resistance, approval atomicity | Probing agents in real containers, concurrency tests |
| System | The complete run, submit and teardown flows | Click CliRunner, Docker, timing harness |
| Usability and accessibility | Contrast ratios, responsive layout, keyboard navigation | Jest contrast test, Playwright layout tool |

### 6.2.1 Functional Testing

Functional testing verified that each function, module and security control behaves as specified. Unit tests checked properties such as a dummy credential never containing the real value and the egress decision table for IP literals and private addresses. Security-focused tests checked that a non-allowlisted connection and DNS-based exfiltration are

blocked, that a tampered in-container Git cannot hide a file from the diff, and that concurrent approve and reject requests resolve to exactly one decision.

### 6.2.2 System Testing

System testing drove the real vaultshield run command end to end with a real container, clone, agent process, trusted capture and teardown, and the same harness executed the 40 timed runs reported in Chapter 7.

## 6.3 Testing Results and Findings

Table 6.2 summarises the results of the three automated suites.

**Table 6.2:** Automated test suite results

| Suite | Tests | Passed | Failed | Pass rate |
|---|---:|---:|---:|---:|
| CLI | 2,080 | 2,078 | 2 | 99.90% |
| Backend | 341 | 341 | 0 | 100.00% |
| Frontend | 283 | 283 | 0 | 100.00% |
| **Total** | **2,704** | **2,702** | **2** | **99.93%** |

No test was skipped. The lowest text contrast was 4.77:1 in the light theme and 5.80:1 in the dark theme, above the 4.5:1 threshold, and the layout report recorded no horizontal overflow across 63 page and width combinations. Table 6.3 lists the main issues found during development and testing.

**Table 6.3:** Issues found and their resolution

| Issue | Resolution | Status |
|---|---|---|
| 54 of 72 losing concurrent decisions received HTTP 500 instead of 409 | Retry on transient transaction errors | Fixed |
| Dummy connection strings kept the real database host name | Host replaced unless provably local | Fixed |
| A missing credential file became an empty directory through Docker's bind mount | File check before any Docker call | Fixed |
| Host listeners reachable from a plain internal bridge | Internal bridge created without a gateway | Fixed |
| A hanging interactive sign-in was not bounded by its deadline | Under investigation on Docker Desktop 29.2.1 | Open |
| The file watcher missed the agent's write in some fast runs | The trusted diff remains the authoritative record | Open |
| The status PATCH route can set approved without an Approval record | Documented MVP limitation; gating planned | Open |

## 6.4 Sample Test Cases

Table 6.4 presents representative test cases, each corresponding to an automated test or to the benchmark harness.

**Table 6.4:** Sample test cases

| ID | Description | Expected and actual result | Status |
|---|---|---|---|
| TC-01 | Sandbox isolation | Probing agent cannot read the host .env, home directory or Docker socket; host checkout untouched in 40 of 40 runs | Pass |
| TC-02 | Secret suppression | No generated dummy value contains its real value; 135 of 135 tests passed | Pass |
| TC-03 | Network blocking | CONNECT to unlisted hosts, IP literals and the host answered 403; direct DNS lookups fail | Pass |
| TC-04 | Off-limits enforcement | Write to a protected path records a violation event, then fails the run | Pass |
| TC-05 | Tamper resistance | A file hidden by a tampered in-container Git still appears on the run branch | Pass |
| TC-06 | Token hygiene | All five credential-leak vectors fail the run and quarantine the diff | Pass |
| TC-07 | Approval gating | submit refuses every non-approved run with no push and no PR | Pass |
| TC-08 | Concurrent decisions | Exactly one winner per trial and exactly one Approval record | Pass |
| TC-09 | Monitoring completeness | Every agent write in the timeline; present in 32 of 40 runs, while the diff recorded 40 of 40 | Partial |
| TC-10 | Hanging sign-in is bounded | Expected to stop within 60 s; took 600.3 s | Fail |

## 6.5 Conclusion on Testing

Of 2,704 automated tests, 2,702 passed, and every security-critical behaviour named in the objectives was exercised against real infrastructure. The two failures affect only the optional interactive sign-in flow. The file watcher's missed events are a gap in supplementary evidence rather than in change capture, because the trusted diff was complete in every run, and the unrestricted status PATCH route is acceptable only within the single-developer trust model of the MVP.

## 6.6 Test Configuration and Analysis of the Two Failed Tests

### 6.6.1 Test Environment and Configuration

Table 6.5 records the configuration under which the suites and the benchmark were run.

**Table 6.5:** Test environment and configuration

| Item | Configuration |
|---|---|
| Hardware | Apple MacBook Air, M4 processor |
| Container runtime | Docker Desktop 29.2.1 (Docker Engine 29.2.1) |
| CLI suite | Python 3.14, pytest 8 with Hypothesis; Docker-backed tests use the vaultshield sandbox image and a real daemon; command: cd cli && pytest -q |
| Backend suite | Python, pytest against a MongoDB replica set in a throw-away test_ database; command: cd backend && pytest -q |
| Frontend suite | Jest with React Testing Library and fast-check; command: cd frontend && npm test |
| Sample repository | https://github.com/octocat/Hello-World.git (public, one README file) |
| Agent for system tests | Deterministic mock agent (Section 7.2) |

The suites were re-run on 1 November 2026 to confirm the reported figures, as shown in Figure 6.1. The backend (341 of 341) and frontend (283 of 283) results were identical. The CLI suite now has 2,084 tests, four added after the recorded campaign; 2,082 passed and the same two failed.

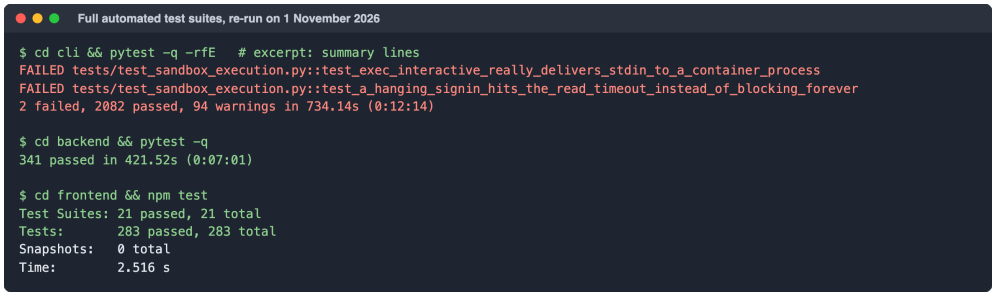



**Figure 6.1:** Re-run of the three automated suites on 1 November 2026

### 6.6.2 Analysis of the Two Failed Tests

Both failing tests are Docker-backed tests in tests/test_sandbox_execution.py, and both exercise the interactive sign-in transport used only when the antigravity agent is run with --interactive-auth. This transport writes a pasted authorisation code to the agent's standard input through the Docker exec socket and redacts it from everything the CLI displays. Figure 6.2 shows both failures re-run individually.

**Failure 1: Hanging Sign-in Not Bounded by Its Deadline.** The test test_a_hanging_signin_hits_the_read_timeout_instead_of_blocking_forever starts a fake sign-in program in a real container that prints the code prompt and then sleeps for 600 seconds.




**Figure 6.2:** The two failing tests re-run individually, with their durations

The test shortens the read deadline to 3 seconds and expects the CLI to give up within 60 seconds with a "no further output" error. Instead, the read blocked for 600.33 seconds until the fake program exited, and because the program then exited normally, the sign-in phase returned no error at all.

The cause is that the read deadline is set as a timeout on the exec socket, and on Docker Desktop 29.2.1 that timeout does not interrupt the blocking read of the attached output stream, so the expected socket timeout is never raised. The planned fix is a watchdog that enforces the deadline independently of the socket and kills the exec when it expires.

**Impact.** The failure affects availability only. A sign-in that stops responding keeps the CLI waiting until the agent process exits or the developer presses Ctrl-C, instead of failing after the deadline. It does not expose the authorisation code or any secret, because the code is still redacted and teardown still runs when the phase ends or is interrupted, which other passing tests verify. It does not affect runs without --interactive-auth, including the benchmark.

**Failure 2: Redacted Echo Does Not Match the Test's Expectation.** The test test_exec_interactive_really_delivers_stdin_to_a_container_process sends a sentinel string to a container process that reads one line and prints RECEIVED:[value]. The test expects the raw sentinel in the output. The output was RECEIVED:[***]. Because redaction replaces only an exact match of the sent value, this shows that the process received exactly the bytes sent and that the echo was then masked as designed. The test predates the redaction, so its expectation is out of date.

**Impact.** None on functionality or security. The output itself proves stdin delivery, and the masking is the intended protection that keeps an authorisation code off the terminal and out of the logs.

**Effect on System Reliability.** The two failures are 0.07% of the 2,704 tests. Neither lies in the isolation, secret-substitution, egress, trusted-capture, token-hygiene or approval tests, all of which passed. Those tests do not use the interactive transport: they drive the sandbox through the ordinary non-interactive exec path that every default run uses, so a defect in the optional sign-in path cannot hide a failure in them or in the default workflow. Failure 1 is a genuine functional defect: the code does not do what it is specified to do, and a user of the optional sign-in flow can observe it. Failure 2 is an outdated test assertion: the code behaves as specified, and only the test's expectation is wrong. Failure 1 is the open issue listed in Table 6.3 and the failing case TC-10 in Table 6.4; its fix is part of the Semester VIII monitoring work. Failure 2 needs only the test's assertion changed to the redacted form.

# Chapter 7

# Result and Discussions

## 7.1 Introduction

This chapter evaluates VaultShield with a timing harness that executed 40 complete vaultshield run invocations against real Docker, a separate baseline of 20 runs of the same agent directly on the host, the security-focused test cases of Chapter 6, the feasibility spike on secret exposure, captures of real executions, and a case study of preserved run directories.

**Scope of the performance results.** Every timing in this chapter was measured with VaultShield's deterministic *mock agent*, not with a real model-backed agent such as Codex, Kiro or Antigravity. The mock agent runs inside the sandbox through the same adapter, PTY and monitoring path as a real agent, prints a few lines and writes one small file, but it makes no model API calls, opens no network connections and finishes in a fraction of a second. The results therefore measure the fixed cost of the VaultShield pipeline itself on one machine and one small repository. They do not show how long a real agent takes, how its output quality changes, or how VaultShield's overhead behaves when an agent edits many files, produces large outputs or sends much traffic through the egress proxy. Those measurements are planned for Semester VIII.

## 7.2 Experimental Setup

The benchmark ran on an Apple MacBook Air with an M4 processor running Docker Desktop 29.2.1, against a small local repository and a stubbed backend, so that network latency to a real backend did not enter the timings. Table 7.1 lists the four configurations; the remaining software configuration is given in Table 6.5.

**Table 7.1:** Benchmark configurations (all with the mock agent)

| Configuration | Runs | Execution | Egress | Screen-shots | Purpose |
|---|---|---|---|---|---|
| Direct on host | 20 | Host clone, no container | None | 0 | Baseline without VaultShield |
| VaultShield default | 20 | Sandbox + trusted capture | Off | 0 | Fixed pipeline overhead |
| VaultShield with egress allowlist | 10 | Sandbox + trusted capture | Allowlist | 0 | Cost of network isolation |

| Configuration | Runs | Execution | Egress | Screen-shots | Purpose |
|---|---|---|---|---|---|
| VaultShield with one screenshot route | 10 | Sandbox + trusted capture | Off | 1 | Cost of browser capture |

The benchmark therefore consists of 60 timed runs: 40 VaultShield runs (20 default, 10 with the egress allowlist and 10 with one screenshot route) and 20 direct-host baseline runs. Every reference in this report to "the 40 runs" means the 40 VaultShield runs; the 20 baseline runs did not use VaultShield and are used only to compute overhead. In every VaultShield run the mock agent wrote the file vaultshield_mock_agent.txt into the clone. After each run the harness checked that the host checkout was unchanged, that the file appeared in diff.json and in the event log, and that no VaultShield container or network remained.

## 7.3 Evaluation Metrics

Table 7.2 defines the metrics used.

**Table 7.2:** Evaluation metrics

| Metric | Definition |
|---|---|
| End-to-end latency | Wall-clock time of one vaultshield run, from command start to exit |
| Stage latency | Time spent in each pipeline stage |
| Overhead | Difference between VaultShield latency and the same agent run directly on the host |
| Isolation effectiveness | Fraction of runs in which the host checkout was unchanged and host files were unreachable |
| Capture and monitoring completeness | Fraction of runs whose diff and event log contained the agent's change |
| Detection of unsafe actions | Fraction of injected unsafe behaviours that were blocked or detected |
| Cleanup reliability | Number of VaultShield containers or networks left after all runs |

## 7.4 Performance Analysis

Table 7.3 reports the stage latency of the default configuration over 20 runs, in seconds, and Figure 7.1 shows the mean of each stage.

**Table 7.3:** Stage latency in the default configuration

| Stage | Mean | Std. dev. | Share |
|---|---:|---:|---:|
| Sandbox creation, clone and install | 0.239 | 0.017 | 17.0% |
| Agent execution and monitoring | 0.413 | 0.033 | 29.4% |
| Trusted capture | 0.651 | 0.053 | 46.4% |
| Token-hygiene scan | 0.019 | 0.008 | 1.4% |
| Teardown | 0.042 | 0.006 | 3.0% |
| Other stages | 0.039 | – | 2.8% |
| **End-to-end total** | **1.403** | **0.075** | **100%** |

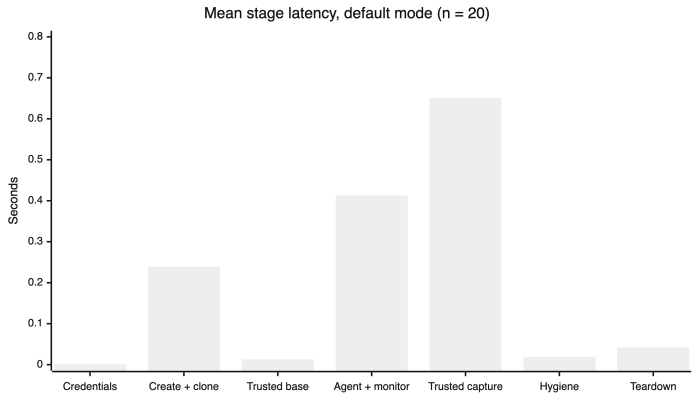



**Figure 7.1:** Mean stage latency in the default configuration

Trusted capture is the most expensive stage because it starts a second container, extracts the work tree and rebuilds the repository; this is the price of a diff the agent cannot falsify. With a real agent, the agent-execution stage would be dominated by model response time, and the trusted-capture and token-hygiene stages would grow with the size of the change, so the stage shares in Table 7.3 apply only to the mock agent's one-file change. Agent execution took 0.413 s against 0.186 s on the host, the difference covering the in-container watcher and Docker's exec API. Table 7.4 and Figure 7.2 compare the configurations.

**Table 7.4:** End-to-end latency by configuration

| Configuration | Runs | Mean time, s |
|---|---:|---:|
| Direct on host: clone and agent | 20 | 0.237 |
| VaultShield default | 20 | 1.403 |
| VaultShield with egress allowlist | 10 | 2.213 |
| VaultShield with one screenshot route | 10 | 3.294 |

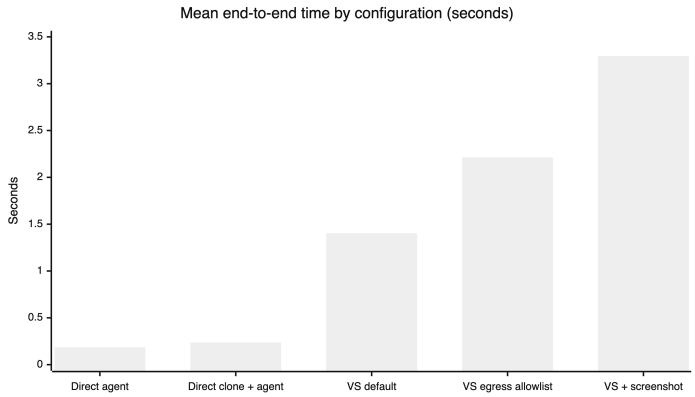



**Figure 7.2:** Mean end-to-end time by configuration

The egress allowlist added 0.810 s, mostly for creating the per-run networks and proxy and removing them at teardown. One screenshot added 1.891 s, almost all of it spent starting the application and waiting for it to answer.

## 7.5 Overhead with and without VaultShield

Against direct execution with a fresh clone, VaultShield's default mode added 1.166 s per run and the egress allowlist mode added 1.976 s with the mock agent. If this overhead stayed fixed, it would be 0.39% and 0.66% of a five-minute agent session respectively. That figure is a projection, not a measurement: it assumes that a real agent's session length is unaffected by the sandbox and that capture and scanning costs do not grow with the change, neither of which has been measured yet. It indicates the order of magnitude of the pipeline's own cost and should not be read as the measured overhead for Codex, Kiro or Antigravity. Section 7.9 places the overhead alongside the qualitative differences (Table 7.7).

## 7.6 Safety Events Captured

Table 7.5 summarises the safety-relevant results across the 40 VaultShield benchmark runs and the security-focused test cases. These are rates over injected scenarios, which show that each control works for the behaviour it was designed against but do not prove the controls are exhaustive.

**Table 7.5:** Safety events and evidence captured

| Scenario | Result |
|---|---|
| Host checkout unchanged after the run | 40 of 40 |
| Agent's file present in diff.json | 40 of 40 |
| File-change event recorded for the agent's file | 32 of 40 |
| Non-allowlisted connections blocked and logged | 4 of 4 |
| Credential-leak vectors detected | 5 of 5 |
| Tampered in-container Git defeated | 2 of 2 |

| Scenario | Result |
|---|---|
| Values in six real .env.sandbox files that are dummies | 30 of 30 |
| Containers and networks left after the campaign | 0 |

## 7.7 Secret Exposure With and Without Protection

The feasibility spike asked a production coding agent to read .env and .env.local and report the exact values of two fake keys. Table 7.6 reports the outcome under four protection configurations.

**Table 7.6:** Secret exposure under four protection configurations

| Config | Protection layers | Verdict |
|---|---|---|
| C0 | None | Both secrets printed |
| C1 | Agent deny rules for .env files | .env.local read into context |
| C2 | C1 plus a hook denying shell access to .env | .env.local read into context |
| C3 | C2 plus real values replaced by a placeholder on disk | Safe |

Agent-level deny rules proved brittle, and only keeping the real value off disk was safe. This is why the sandbox receives dummy values and why the Protect module moves values into the macOS Keychain for agents run outside the sandbox. Unlike the benchmark, this experiment used a real production coding agent, but a single agent and a single prompt, so it shows that deny rules can fail rather than how often they fail.

## 7.8 Execution Evidence from a Live Run

To complement the benchmark tables, the complete system was executed on 1 November 2026 against the running Docker Compose stack and the public repository octocat/Hello-World, using the mock agent with the egress allowlist enabled. The figures below are captures of those real executions. Terminal output was saved from the commands and rendered in a terminal frame without editing; where an output was long, the figure shows an excerpt of selected lines, marked as such in the figure. The dashboard images are screenshots of the live dashboard showing the uploaded run.

Figure 7.3 shows the run itself. The run was registered with the backend before the sandbox was created, the dummy credential file was built from the developer's .env, the sandbox was moved onto the egress-isolated network, the change was committed to the run branch in the capture helper, and the sandbox was torn down before the evidence was uploaded. The whole command took 6.89 s of wall-clock time including the clone from GitHub, the backend upload and one screenshot.

Figure 7.4 shows the artifacts that run produced. Every value in .env.sandbox is a dummy of the same shape as the real value, including a database URL whose credentials

vaultshield run with the egress allowlist (mock agent)

```
$ vaultshield run --agent=mock --prompt="add a file" --egress allowlist \
    --repo-url="https://github.com/octocat/Hello-World.git" --project 6ac8a852d5409c14bd3543c2
Run 20261101-180424-c5458c45adecdf4a: artifacts in /Users/aayush/Codes/VaultShield/.vaultshield/runs/20261101-180424-c5458c45adecdf4a
Run 20261101-180424-c5458c45adecdf4a registered with http://localhost:8000.
Sandbox_Credential_File built from /Users/aayush/Codes/VaultShield/.env at /Users/aayush/Codes/VaultShield/.vaultshield/runs/20261101-180424-c5458c45adecdf4a/.env.sandbox.
Sandbox created for run 20261101-180424-c5458c45adecdf4a (cloned https://github.com/octocat/Hello-World.git).
Sandbox for run 20261101-180424-c5458c45adecdf4a isolated: egress allowlist for agent mock.
Captured 1 screenshot(s) into /Users/aayush/Codes/VaultShield/.vaultshield/runs/20261101-180424-c5458c45adecdf4a/screenshots
Committed run 20261101-180424-c5458c45adecdf4a to branch vaultshield/run-20261101-180424-c5458c45adecdf4a in the capture helper; structured diff written to /Users/aayush/Codes/VaultShield/.vaultshield/runs/20261101-180424-c5458c45adecdf4a/diff.json
Run 20261101-180424-c5458c45adecdf4a: branch bundle written to /Users/aayush/Codes/VaultShield/.vaultshield/runs/20261101-180424-c5458c45adecdf4a/branch.bundle (561 bytes).
Sandbox torn down for run 20261101-180424-c5458c45adecdf4a.
Run 20261101-180424-c5458c45adecdf4a complete. Event_Log written to /Users/aayush/Codes/VaultShield/.vaultshield/runs/20261101-180424-c5458c45adecdf4a/events.jsonl
Run 20261101-180424-c5458c45adecdf4a uploaded to http://localhost:8000
real 6.89
user 0.78
sys 0.35
exit=0
```

**Figure 7.3:** A real vaultshield run with the egress allowlist and the mock agent

and host were replaced, and the event log records the agent's output lines and the watcher's independent file-change event. The structured diff contains exactly the one file the agent created.

Artifacts written by the run

```
$ cat .env.sandbox            # dummy values only
VAULTSHIELD_MONGODB_URL=mongodb+srv://dummy_user:dummy_password@localhost/?retryWrites=true&w=majority
VAULTSHIELD_MONGODB_NAME=dummy-value
SECRET_KEY=dummy-value
DEBUG=dummy-value
ALLOWED_HOSTS=dummy-value
GEMINI_API_KEY=dummy-value
GROQ_API_KEY=z6uyz8QLg1ajKg9BC9ZKJAqDIOmvIZOwicsbOb1CGETWdwEKRk8OFdzD

$ cut -c1-150 events.jsonl     # lines cut at 150 characters
{"event_type": "output", "timestamp": "2026-11-01T18:04:26.582646+00:00", "payload": "[mock-sandbox] cwd=/workspace/repo"}
{"event_type": "output", "timestamp": "2026-11-01T18:04:26.582706+00:00", "payload": "[mock-sandbox step=0] simulated output for prompt='add a file'"}
{"event_type": "output", "timestamp": "2026-11-01T18:04:26.637693+00:00", "payload": "[mock-sandbox step=1] simulated output for prompt='add a file'"}
{"event_type": "output", "timestamp": "2026-11-01T18:04:26.693660+00:00", "payload": "[mock-sandbox step=2] simulated output for prompt='add a file'"}
{"event_type": "output", "timestamp": "2026-11-01T18:04:26.750056+00:00", "payload": "[mock-sandbox] wrote vaultshield_mock_agent.txt"}
{"event_type": "output", "timestamp": "2026-11-01T18:04:26.750116+00:00", "payload": "[mock-sandbox] simulated final message for prompt='add a file'"}
{"event_type": "file_change", "timestamp": "2026-11-01T18:04:26.862517Z", "payload": "created: /workspace/repo/vaultshield_mock_agent.txt"}

# diff.json, pretty-printed with Python's json module, lines cut at 150 characters
{
 "files": [
  {
   "path": "vaultshield_mock_agent.txt",
   "additions": 1,
   "deletions": 0,
   "patch": "diff --git a/vaultshield_mock_agent.txt b/vaultshield_mock_agent.txt\nnew file mode 100644\nindex 0000000..f59c895\n--- /dev/null\n+++ b/
  }
 ]
}
```

**Figure 7.4:** Run artifacts: dummy credential file, event log and structured diff

Figure 7.5 and Figure 7.6 show the same run in the live dashboard, awaiting review, with its timeline and diff. Both are unedited screenshots cropped to the run panel; the project sidebar was removed for legibility.

Figure 7.7 shows the approval gate and the cleanup check. Submitting the run while

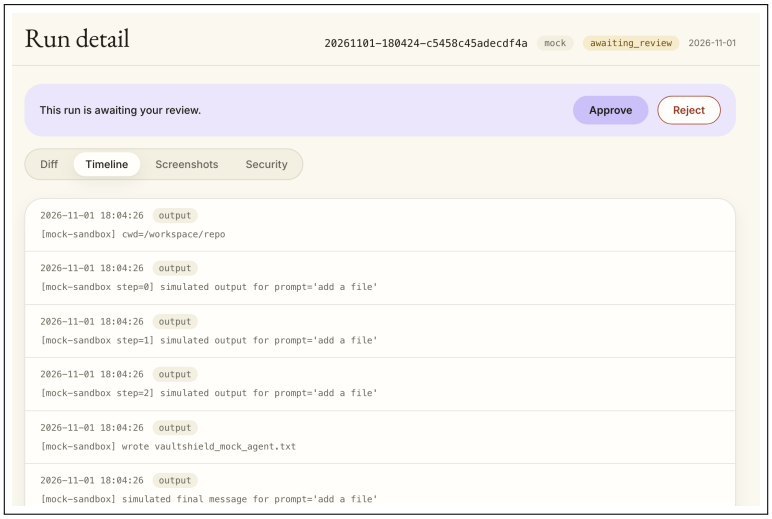



**Figure 7.5:** Live dashboard: Timeline tab of the real run




**Figure 7.6:** Live dashboard: Diff tab of the real run, awaiting review

it was still awaiting review was refused before any side effect, no VaultShield sandbox, capture helper or proxy container remained, the only remaining network was the Compose stack's own, and the agent's file did not exist in the host checkout.

Approval gate and cleanup check after the run

```
$ vaultshield submit 20261101-180424-c5458c45adecdf4a
Error: Run 20261101-180424-c5458c45adecdf4a is 'awaiting_review', not 'approved', so it cannot be submitted. Nothing was pushed
and no pull request was opened.
exit=1

$ docker ps -a --filter name=vaultshield-run --filter name=vaultshield-capture --filter name=vaultshield-egress
NAMES     STATUS

$ docker network ls --filter name=vaultshield
NAME                 DRIVER
vaultshield_default   bridge

$ ls ../vaultshield_mock_agent.txt   # run from cli/: the host checkout root
ls: ../vaultshield_mock_agent.txt: No such file or directory
```

**Figure 7.7:** Approval gate refusing an unapproved run, and no leftover containers or host changes

Figure 7.8 shows the isolation probe, a standalone script that creates a real sandbox through the same class the run command uses and tries what a hostile agent might try. The host .env and the Docker socket were absent, the working directory was the sandbox clone, and only the dummy credential file was readable. The home directory listed by the probe is the container's own /root, which contains only image defaults.

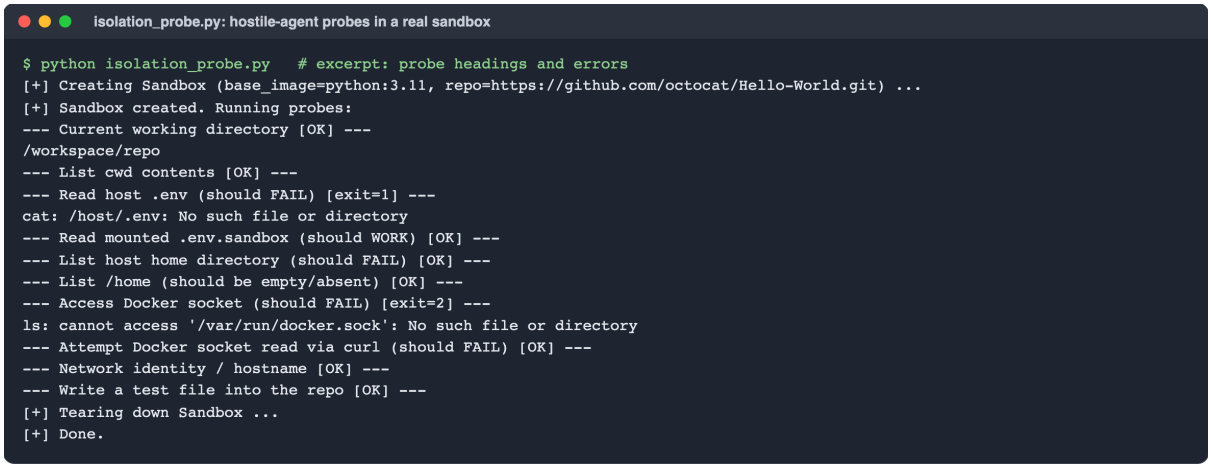



**Figure 7.8:** Isolation probe run inside a real sandbox

Figure 7.9 shows the Docker-backed security tests run against real containers: a blocked connection is blocked and recorded, DNS exfiltration is blocked, all five token-leak vectors are detected, and a tampered in-container Git cannot hide a file from the branch or diff. All 15 tests passed in 34.4 s.

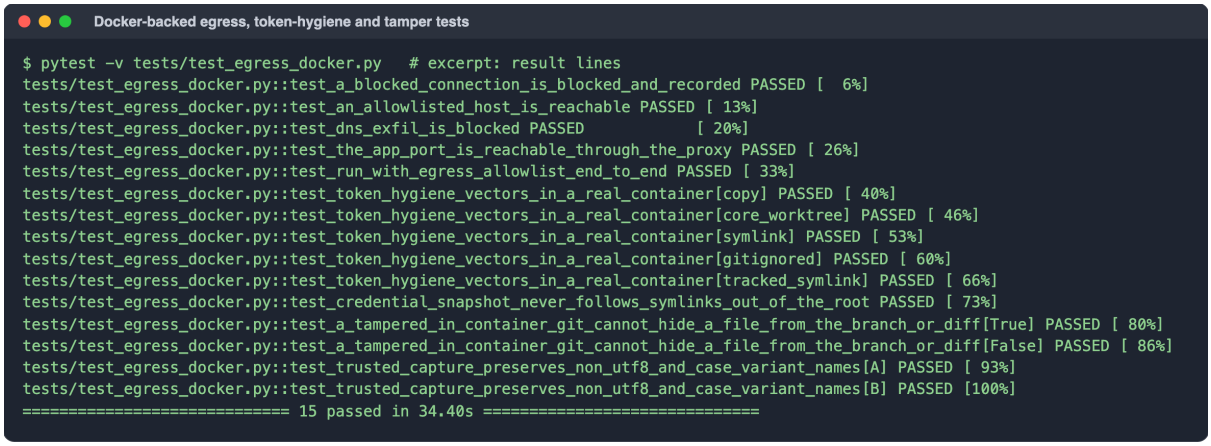



**Figure 7.9:** Docker-backed egress, token-hygiene and tamper-resistance tests

## 7.9 Comparison with Existing Approaches

Table 7.7 compares the outcome of the same threats under three approaches: running the agent directly on the host, relying on agent-native protection (deny rules, OS sandbox and approval prompts, as documented for Claude Code and Codex CLI [16], [18]), and VaultShield. The direct-execution and VaultShield columns, and the deny-rule result, were measured in this project; the remaining agent-native entries come from vendor documentation and were not measured by us.

**Table 7.7:** Outcome of the same threats under three approaches

| Threat or property | Direct on host (measured) | Agent-native protection | VaultShield (measured) |
|---|---|---|---|
| Agent reads a secret in the project .env | Both secrets printed (C0) | Deny rules bypassed via .env.local (C1, C2, measured) | No real value present; 30 of 30 values dummies |
| Agent modifies the developer's checkout | Yes | Yes, edits land in the workspace by design | Host unchanged in 40 of 40 runs |
| Outbound connection to an unlisted host | Allowed | Blocked by default in Codex CLI; configurable in Claude Code | Blocked and logged, 4 of 4, when the allowlist is on |
| Agent hides a change by tampering with Git | Possible | Not addressed in cited documentation | Defeated, 2 of 2 |
| Agent's credential leaks into its output | Not checked | Not addressed in cited documentation | Detected, 5 of 5 vectors |
| Change reaches the repository without review | Yes, if the developer commits | Per-action prompts; no held change | Refused until approved (Fig. 7.7) |
| Works with several agents | – | No, built into one agent | Yes, four adapters |
| Fixed time cost per run (mock agent) | 0.237 s | Not measured | +1.166 s, or +1.976 s with egress |
| Containers or networks left behind | – | Not measured | 0 after all runs |

The comparison shows where VaultShield adds protection that the other approaches do not provide, and also that agent-native sandboxes already block the network by default where VaultShield requires opting in. The two kinds of control are complementary: an agent's own sandbox can still run inside a VaultShield sandbox.

## 7.10 Case Study: Analysis of Real Runs

The repository preserves five complete mock-agent runs from development. In run 20261009-091725-ff669fcd7dd00b2b, the event log records seven events: the agent confirms that its working directory is the sandbox clone, narrates its work over 157.9 ms, and the in-container watcher then independently reports the created file 106.5 ms after the agent's write. The diff contains exactly that file, and the screenshot shows the sandbox clone's directory listing. In one of the five runs, however, the event log has no file-change event even though the diff and screenshot both show the file, the same gap seen in 8 of the 40 benchmark runs. The event log is therefore corroborating evidence of behaviour, while the trusted diff is the authoritative record of what changed.

## 7.11 Overall Discussion and Limitations

With the mock agent, separating the execution of an agent from trust in its output cost about 1.2 seconds per run, or about 2.0 seconds with network allowlisting. In exchange, every evaluated run left the developer's checkout untouched, kept real secrets out of the agent's environment, recorded an event log, captured the change on a branch the agent could not falsify and held it behind an approval state machine. Table 7.8 lists the main residual risks.

**Table 7.8:** Residual risks and limitations

| Limitation | Consequence | Planned improvement |
|---|---|---|
| Shared-kernel container isolation; default sandbox runs as root | An exploit could reach the host | Default hardening, gVisor or microVMs |
| Egress unrestricted unless enabled | Repository content could be exfiltrated | Default-deny egress |
| Host-based allowlist | Uploads to an allowlisted service are possible | Path-aware filtering |
| Status PATCH can set approved; no multi-user model | A token holder can skip review | Gated transitions, reviewer roles |
| Polling file watcher misses some writes | Timeline may omit a change | Diff cross-check, event-driven watcher |
| Diff tab shows counts, not patch text | Line-level review needs diff.json | Inline patch rendering |

The evaluation itself is limited. All timing figures come from the mock agent on one machine and one small repository, so they describe the pipeline's fixed overhead rather than the experience of a real agent, and they must not be generalised to Codex, Kiro, Antigravity or any other model-backed agent. The safety results are rates over injected scenarios and do not measure resistance to an adaptive adversary. Evaluation with real agents on representative repositories is part of the Semester VIII plan.

## 7.12 Chapter Summary

With the mock agent, the default pipeline completed in 1.403 s on average, adding 1.166 s to direct execution, with trusted capture the largest stage; real-agent timings remain to be measured. Isolation, change capture and cleanup succeeded in every evaluated run, all injected unsafe behaviours were blocked or detected, and every value in six real sandbox credential files was a dummy. The file watcher recorded the agent's change in 80% of runs, confirming the trusted diff as the authoritative record.

# Chapter 8

# Conclusion

VaultShield separates running an AI coding agent from trusting what it produces. The agent receives a disposable Docker container, a fresh clone and only dummy credentials; its output, file changes and network decisions are recorded in an event log; its change is captured as one commit on a dedicated branch by a separate hardened container; and the change can become a pull or merge request only after a human approves it in the review dashboard. The system was built as a Python CLI, a Django REST Framework backend on MongoDB with Celery workers, and a React dashboard, totalling about 18.1 thousand logical lines of product code. Table 8.1 assesses the outcome against each objective.

**Table 8.1:** Achievement of objectives

| Objective | Evidence | Outcome |
|---|---|---|
| 1. Universal agent execution platform | Mock, codex, kiro and antigravity agents run behind one adapter interface | Achieved |
| 2. Agentic AI Orchestrator | Workflow, components, security constraints and evaluation plan designed (Section 3.5); implementation scheduled for Semester VIII | Planned for Semester VIII |
| 3. Secure sandboxed execution | Host checkout and .env unchanged in 40 of 40 runs; dummy credentials and one-shot clone token | Achieved |
| 4. Monitoring and security analysis | Event log and diff for every run; violations recorded; Celery security scan | Achieved, with a file-watcher gap |
| 5. Governance and human approval | Dashboard review and decision controls; submit refuses non-approved runs | Achieved |

Objectives 1, 3, 4 and 5 were achieved in Semester VII. Objective 2, the Agentic AI Orchestrator, was not implemented in Semester VII; its workflow and expected functionality are specified in Section 3.5 and its implementation is scheduled for Semester VIII. Two further gaps remain: in the single-developer MVP a token holder can bypass the dashboard through an unrestricted status update, and the polling file watcher missed some writes in very short runs, although the trusted diff captured every change. The project contributes a working implementation of the governance practices recommended for agentic systems [9], [10], engineering techniques such as shape-preserving credential substitution, a one-shot clone credential and trusted capture, and evidence that the pipeline's own fixed cost is about 1.2 seconds per run with a mock agent, while agent-level deny rules alone do not protect secrets. Real-agent performance has not yet been measured.

# Chapter 9

# Future Scope

The limitations identified in this report define the future work. The first four items are the committed Semester VIII enhancements; the remaining items are longer-term work.

## 9.1 Semester VIII Enhancements

1. **Agentic AI Orchestrator:** implement the design in Section 3.5, which scans the project, asks clarifying questions before execution and passes a sanitised, enriched prompt to the agent, then measure on real agents whether it raises task success and approval rates and reduces hallucinated files, functions and packages. This completes Objective 2.
2. **Stronger Sandbox Isolation:** run the sandbox by default as a non-root user with all capabilities dropped, no-new-privileges and CPU, memory and process limits; make egress default-deny; and support gVisor or Firecracker microVM runtimes for a boundary that does not share the host kernel [12], [13]. This addresses threats T3, T7 and T8 in Table 3.3.
3. **Improved Monitoring:** replace the polling file watcher, which missed 8 of 40 short writes, with an event-driven watcher; cross-check the event log against the trusted diff and flag any changed file missing from the timeline; stream events live so a reviewer can interrupt a run [9]; render patch text in the Diff tab; and enforce the interactive sign-in deadline with a watchdog (Section 6.6.2).
4. **Role-Based Approval:** add separate developer and reviewer accounts, role-based access control, decision attribution and optional multiple approvals, and route every status transition through the gated approval endpoint, so that the approval gate becomes a separation-of-duties control rather than a single-user workflow control. This addresses threat T9.

## 9.2 Further Future Work

- **Policy Engine:** express rules such as required approvals for infrastructure files or approved dependency lists declaratively, with path-aware egress filtering.
- **Cost and Resource Controls:** record per-run token usage and enforce per-project budgets and sandbox time limits.
- **Multi-Repository and Parallel Runs:** capture one branch per repository in a single run, and let several agents attempt the same task for the reviewer to compare.
- **IDE Integration:** start runs and review them from the editor.
- **Evidence Quality:** scan screenshots for credentials.
- **Broader Evaluation:** extend the Semester VIII real-agent evaluation to a subset of SWE-bench [2] and to adaptive red-team attacks.

# Bibliography


[1] Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., et al., "Evaluating Large Language Models Trained on Code," arXiv preprint arXiv:2107.03374, 2021.

[2] Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K., "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?," International Conference on Learning Representations, 2024.

[3] Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., and Press, O., "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering," Advances in Neural Information Processing Systems, 2024.

[4] Wang, X., Li, B., Song, Y., Xu, F. F., Tang, X., Zhuge, M., et al., "OpenHands: An Open Platform for AI Software Developers as Generalist Agents," International Conference on Learning Representations, 2025.

[5] Ruan, Y., Dong, H., Wang, A., Pitis, S., Zhou, Y., Ba, J., Dubois, Y., Maddison, C. J., and Hashimoto, T., "Identifying the Risks of LM Agents with an LM-Emulated Sandbox," International Conference on Learning Representations, 2024.

[6] Debenedetti, E., Zhang, J., Balunović, M., Beurer-Kellner, L., Fischer, M., and Tramèr, F., "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents," Advances in Neural Information Processing Systems, Datasets and Benchmarks Track, 2024.

[7] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., and Fritz, M., "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," ACM Workshop on Artificial Intelligence and Security, 2023.

[8] Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., and Karri, R., "Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions," IEEE Symposium on Security and Privacy, 2022.

[9] Shavit, Y., Agarwal, S., Brundage, M., et al., "Practices for Governing Agentic AI Systems," OpenAI White Paper, 2023.

[10] Chan, A., Ezell, C., Kaufmann, M., Wei, K., Hammond, L., Bradley, H., et al., "Visibility into AI Agents," ACM Conference on Fairness, Accountability, and Transparency, 2024.

[11] Combe, T., Martin, A., and Di Pietro, R., "To Docker or Not to Docker: A Security Perspective," IEEE Cloud Computing, 2016.

[12] Agache, A., Brooker, M., Florescu, A., Iordache, A., Liguori, A., Neugebauer, R., Piwonka, P., and Popa, D.-M., "Firecracker: Lightweight Virtualization for Serverless Applications," USENIX Symposium on Networked Systems Design and Implementation, 2020.

[13] Young, E. G., Zhu, P., Caraza-Harter, T., Arpaci-Dusseau, A. C., and Arpaci-Dusseau, R. H., "The True Cost of Containing: A gVisor Case Study," USENIX Workshop on Hot Topics in Cloud Computing, 2019.

[14] Bacchelli, A., and Bird, C., "Expectations, Outcomes, and Challenges of Modern Code Review," International Conference on Software Engineering, 2013.

[15] Boehm, B. W., "Software Engineering Economics," Prentice-Hall, 1981.

[16] Anthropic, "Configure the Sandboxed Bash Tool," Claude Code documentation, accessed 1 November 2026.

[17] Anthropic, "Making Claude Code Safer with Sandboxing," Anthropic, accessed 1 November 2026.

[18] OpenAI, "Agent Approvals and Security – Codex," OpenAI developer documentation, accessed 1 November 2026.

[19] GitHub, "Responsible Use of GitHub Copilot Cloud Agent on GitHub.com," GitHub Docs, accessed 1 November 2026.

[20] Docker, "Get Started with Local Docker Sandboxes," Docker documentation, accessed 1 November 2026.

[21] OpenHands, "Runtime Architecture," OpenHands documentation, accessed 1 November 2026.